UNIVERSIDAD DE EXTREMADURA

ESCUELA DE INGENIERÍAS INDUSTRIALES

MODELADO DEL SISTEMA VESTIBULAR HUMANO E INTEGRACIÓN EN UN SIMULADOR PARA EL ESTUDIO DEL CONTROL DE ORIENTACIÓN Y BALANCE

TRABAJO PRESENTADO PARA OPTAR AL TITULO DE

MÁSTER UNIVERSITARIO EN INGENIERÍA INDUSTRIAL

POR

ÁNGEL CANELO CALDERA

BADAJOZ, SEPTIEMBRE DE 2017

TRABAJO FIN DE MÁSTER

Autor:

ÁNGEL CANELO CALDERA

Director o Directores:

INÉS TEJADO BALSERA, BLAS MANUAL VINAGRE JARA

Tribunal de evaluación:

SANTIAGO SALAMANCA MIÑO

FRANCISCO JAVIER ALONSO SÁNCHEZ

MARÍA PILAR MERCHÁN GARCÍA

ÍNDICE

1 INTRODUCCIÓN 3
1.1 Objetivo del trabajo 3
1.2 El sistema vestibular humano 3
1.2.1 El vestíbulo 3
1.2.2 Los canales semicirculares 5
1.3 Estructura del documento 7
2 MODELADO CINEMÁTICO 8
2.1 Descripción del modelo cinemático 8
2.1.1 Vectores de velocidad angular 10
2.1.2 Vectores de aceleración angular 11
2.1.3 Vectores de aceleración lineal 12
2.2 El modelo cinemático en Matlab/Simulink 13
3 MODELADO Y SIMULACIÓN DEL SISTEMA VESTIBULAR EN MATLAB/SIMULINK 16
3.1 Modelo dinámico 16
3.1.1 Modelo canales semicirculares 16
3.1.2 Modelo otolitos 19
3.2 Implementación en matlab/simulink 21
3.2.1 Subsistema de los otolitos 22
3.3 Simulaciones 23
3.3.1 Simulación para velocidad constante de la silla 23
3.3.2 Simulación para velocidad trapezoidal de la silla 30
3.3.3 Simulación para velocidad senoidal de la silla 33
4 MODELADO Y SIMULACIÓN DEL ROBOT EN SIMULINK/SIMSCAPE 37
4.1 Creación de piezas en SolidWorks 37
4.2 Implementación en simulink/simscape 39
4.2.1 Modelado del motor de DC 40
4.2.2 Modelo completo 42
4.3 Simulaciones 46
5 INTEGRACIÓN DE MODELOS: INTERFAZ GRÁFICA DE USUARIO 51
6 CONCLUSIONES 52
7 BIBLIOGRAFÍA 53
8 ANEXO: CÓDIGOS DE MATLAB 54
8.1 constantesvestibular.m 54
8.2 MATLAB Function1 (figura 2.5) 55
8.3 MATLAB Function (figura 2.6) 55
8.4 MATLAB Function2 (figura 2.7) 56

8.5 MATLAB Function3 (figura 2.8) 57

8.6 constantes_robot.m 58

8.7 interfaz_grafica.m 59

8.8 modelo_vestibular.m 66

8.9 robot_simulacion.m 67

8.10 DCmotor.m 68

1 INTRODUCCIÓN

1.1 Objetivo del trabajo

El objetivo del presente trabajo es realizar un modelado y simulación del sistema vestibular humano, obteniendo las respuestas de las diferentes partes que componen este sistema (canales semicirculares y otolitos) ante estímulos de aceleraciones/velocidades lineales y angulares que experimentaría la cabeza de una persona a la que se realizara dicha estimulación.

Posteriormente se utilizará como sensor el modelo del canal semicircular humano obtenido, para realizar un control de posición angular de un robot de tres grados de libertad que emula los giros de la cabeza sobre los tres ejes cartesianos (alabeo, cabeceo y guiñada).

1.2 El sistema vestibular humano

El oído interno (Figura 1.1) está dividido en dos partes [4]: la cóclea que sirve a la función auditiva y el sistema vestibular, que contiene los sensores que aportan la información de la orientación del cuerpo y el equilibrio en las tres dimensiones del espacio.

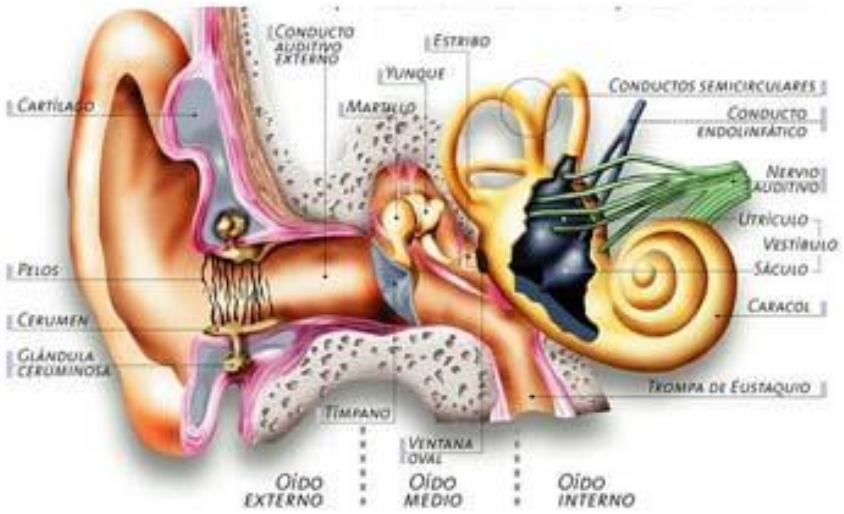


Figura 1.1 Fisiología del oído interno.

Cualquier movimiento del cuerpo es detectado por el sistema vestibular, codificada como una señal eléctrica y enviada al cerebro a través del nervio vestibular. El cerebro luego integra las entradas vestibular, visual y somatosensorial para estimar la orientación y movimiento del cuerpo y en consecuencia, los movimientos del cuerpo para mantener el equilibrio.

El sistema vestibular es una cavidad cuboidea situada en el oído interno, concretamente en el centro del laberinto óseo y que se relaciona con el oído medio por las ventanas oval y redonda. Está formado por el vestíbulo y los conductos semicirculares.

1.2.1 El vestíbulo

En su interior se distinguen dos estructuras membranosas [3]: el utrículo y el sáculo.

En el utrículo y el sáculo se encuentran unos órganos receptores periféricos denominados máculas (Figura 1.2), que están integrados por células receptoras sensoriales ciliadas, las cuales están recubiertas por una membrana horizontal sobre la que hay una serie de cristales de carbonato cálcico que reciben el nombre de otolitos, los cuales son muy susceptibles a las aceleraciones o desaceleraciones lineales en los tres planos del espacio. El estímulo más efectivo para estos receptores es la aceleración lineal producida en el plano paralelo a la mácula.

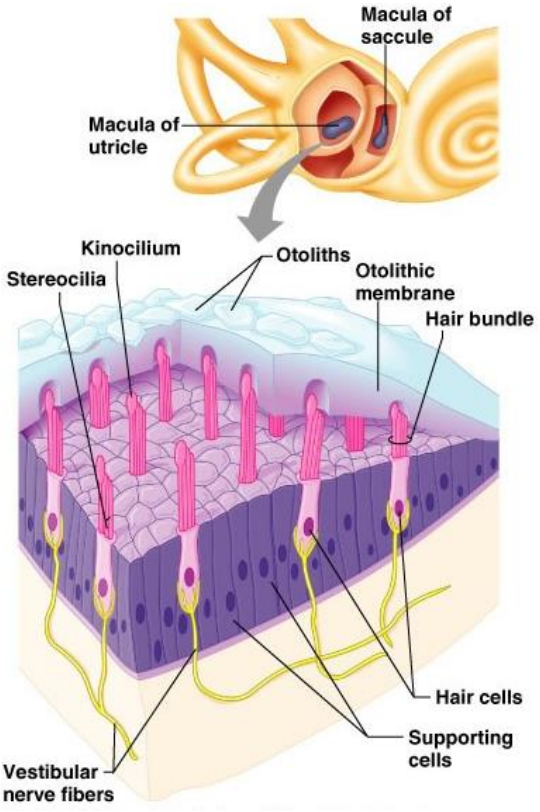


Figura 1.2 Mácula del utrículo y el sáculo.

Considerando al ser humano en bipedestación, las máculas del sáculo se encuentran situadas en un plano vertical y captan de forma eficaz las aceleraciones de los movimientos cefálicos de ascenso y descenso, y por lo tanto de las fuerzas gravitatorias. Al sáculo, además de su función como captor de aceleraciones verticales, se le atribuye una función inmunoprotectora del laberinto, observándose en él abundantes linfocitos. Por otro lado, las máculas del utrículo, al estar situadas en un plano horizontal, capta las aceleraciones lineales laterales y ventro-dorsales así como las inclinaciones de la cabeza.

Por lo tanto, las aceleraciones o desaceleraciones lineales en los tres planos del espacio son el estímulo más eficaz detectado por las máculas otolíticas. Son las células ciliadas de las máculas las encargadas de transformar la energía mecánica, producida por el movimiento, en señales nerviosas.

Durante la aceleración de la cabeza, la membrana otolítica se mueve con respecto a las células ciliadas maculares, produciendo una deflexión de los cilios, acercando estos al quinocilio y provocando una excitación de las mismas. Así, el acercamiento de los estereocilios produce la apertura de los canales de K+, en presencia de Ca2+, produciendo la despolarización hacia su polo basal y la liberación del contenido neuromediador de las vesículas que rodean la barrera sináptica.

De este modo, la flexión de los estereocilios hacia el quinocilio, provoca un aumento de la tasa de estimulación neural. Este aumento de la tasa de estimulación es proporcional a la magnitud del estímulo.

Al alcanzar el movimiento una velocidad constante, no existe ya un desfase entre el movimiento de cilios ni excitación de las células. Cuando el movimiento de la cabeza se desacelera, de nuevo existe un desfase entre el movimiento de las células maculares y de la membrana otolítica, sufriendo los cilios una deflexión en sentido contrario al quinocilio y por tanto una reducción en la liberación de aminas neurotransmisoras y una reducción en la tasa de estimulación neural hasta que la desaceleración cesa.

Las células ciliadas de los receptores otolíticos, no sólo están activas en movimiento sino que mantienen una actividad eléctrica espontánea en reposo, constante e intensa, existiendo una descarga continua de potenciales de acción en las fibras de los nervios vestibulares. Esta actividad continua está producida por el efecto excitador permanente de la fuerza de la gravedad sobre las máculas, sufriéndose una desaferenciación en la ingravidez.

1.2.2 Los canales semicirculares

Son tres, uno horizontal y dos verticales [3]. Son tubos cilíndricos que forman dos tercios de una circunferencia y están orientados en los tres planos del espacio, de manera que el plano de cada uno de ellos forma con el de los otros dos un ángulo de 90 grados (Figura 1.3). De esta forma, son capaces de detectar los movimientos de aceleración angular en los 3 planos del espacio y los componentes vectoriales derivados de estos. El canal semicircular lateral (también llamado horizontal o externo) se encuentra situado en el plano horizontal, cuando el sujeto se encuentra en bipedestación y con la cabeza alineada con el tronco. Esta posición es teórica pues realmente se encuentra levemente inclinado 25º respecto a la horizontal. El canal semicircular superior o anterior se sitúa en el plano frontal, casi perpendicular al eje del peñasco y el canal semicircular posterior se sitúa en un plano sagital.

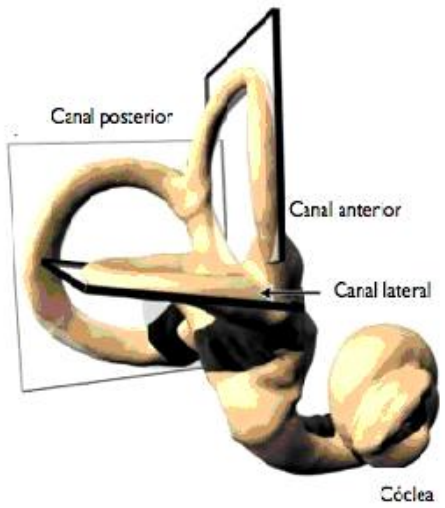


Figura 1.3 Canales semicirculares.

Los canales semicirculares desembocan en el vestíbulo por sus dos extremos, de los que uno, denominado la ampolla, tiene doble diámetro que el otro y es donde residen los receptores de movimiento angular, las crestas ampulares (Figura 1.4). Estas crestas tienen forma de silla de montar, situándose las células neurosensoriales en las dos vertientes de dicha silla, sustentadas por las células de sostén y cubiertas por una sustancia gelatinosa rica en mucopolisacáridos, llamada cúpula, que cierra herméticamente el espacio entre la cresta y el techo de la ampolla. Estas células neurosensoriales son células ciliadas compuestas de 50-100 estereocilios y un quinocilio que se sitúa en el vértice. El quinocilio, se encuentra en la situación cupular más próxima al utrículo en el canal semicircular horizontal y en la posición más alejada en los otros dos canales.

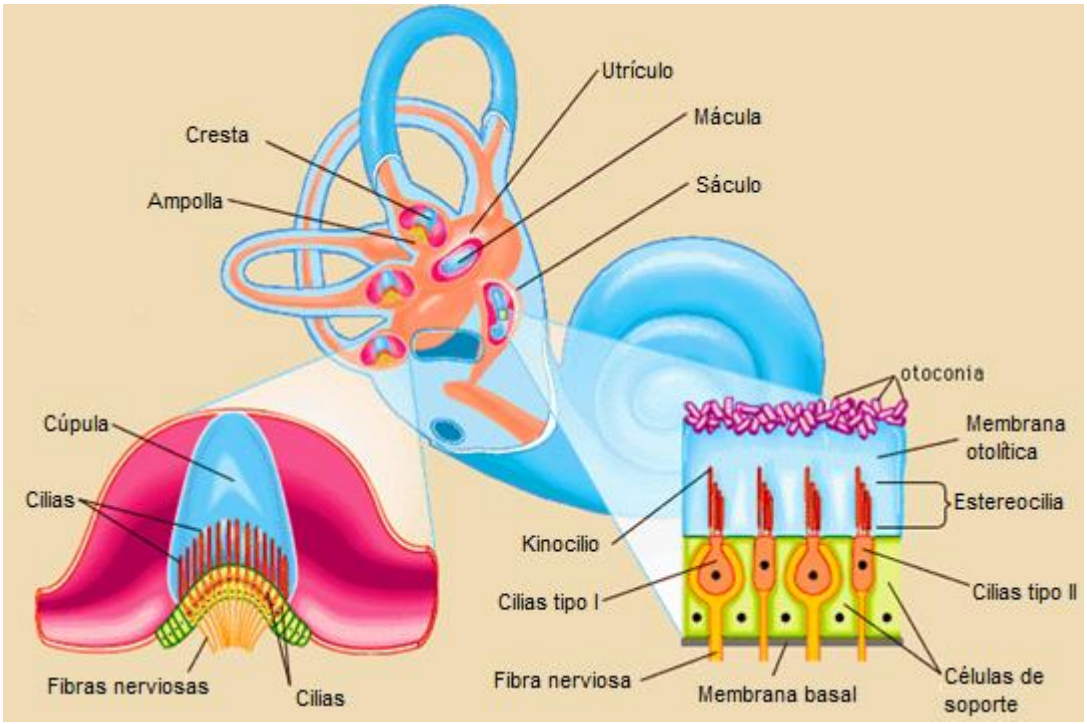


Figura 1.4 Interior de los canales semicirculares.

Los canales semicirculares están llenos de un líquido prístino llamado endolinfa. El factor de estimulación de los canales semicirculares son las aceleraciones angulares, bien por giro de la cabeza o bien rotación de todo el cuerpo. Estas aceleraciones desplazan de forma relativa la endolinfa a lo largo del canal semicircular, en sentido contrario al giro. Así, si se gira la cabeza hacia la derecha, se desplazarían ambos conductos semicirculares, que detectarían el movimiento. El canal semicircular horizontal derecho se desplazaría hacia la derecha, mientras la endolinfa quedaría fija chocando contra la cúpula que se flexiona hacia el utrículo (corriente ampulípeta). La cúpula y la endolinfa ocasionan un movimiento de sentido relativo contrario al giro de la cabeza, desplazando los estereocilios hacia el quinocilio, que está más próximo al utrículo, originando una despolarización. Así, la cúpula, junto con la endolinfa, funciona como un acoplador entre la aceleración angular de la cabeza y las células sensoriales.

Por el contrario, en el lado izquierdo, el mismo movimiento, hace que la endolinfa choque contra la cúpula, desplazándola en sentido opuesto (corriente ampulífega), generando una hiperporalización.

Cuando cesa la aceleración y la velocidad se hace constante, la endolinfa adquiere la misma velocidad que el canal semicircular, gracias a la fricción entre ambos. Al ir adquiriendo la endolinfa igual velocidad que el canal, la cúpula va paulatinamente situándose en la posición de reposo. Por otro lado, cuando se produce una deceleración, la endolinfa mantiene la fuerza de inercia se mueve en sentido contrario a la deceleración.

El proceso de flexión y deflexión de los estereocilios hacia y desde el quinocilio sigue un proceso similar al experimentado en las células ciliadas de las máculas otolíticas.

1.3 Estructura del documento

El presente trabajo se ha estructurado en una memoria, que cuenta con seis capítulos, bibliografía y anexos. Para ofrecer una visión global del documento a continuación se explica de forma breve lo que se trata en cada capítulo.

El presente capítulo se presenta el objetivo del trabajo y una visión global del sistema vestibular.

El capítulo dos explica el modelado cinemático que se plantea y su implementación en Matlab/Simulink.

El capítulo tres trata sobre los modelos dinámicos utilizados y su integración en Matlab junto al modelo cinemático.

En el capítulo cuatro se lleva a cabo la descripción del robot y su simulación en Matlab/Simulink.

El capítulo cinco integra en una interfaz gráfica las simulaciones del modelo del sistema vestibular por un lado y las simulaciones del robot por otro lado.

El capítulo seis plantea las conclusiones obtenidas de este trabajo.

2 MODELADO CINEMÁTICO

En este trabajo se propone una simulación de una estimulación del sistema vestibular para observar el comportamiento de los diferentes elementos que lo componen.

Para realizar esta estimulación se tiene un modelo cinemático, cuyas salidas (velocidades angulares y aceleraciones lineales) serían las entradas al modelo dinámico del sistema vestibular, separado en dos, el modelo de los canales semicirculares por un lado (estimulado por las velocidades angulares obtenidas) y el modelo de los otolitos (mácula otolítica) por otro (estimulado por las aceleraciones lineales obtenidas).

El modelo dinámico de los canales semicirculares y de los otolitos se desarrollará en el capítulo 3 de este trabajo.

A continuación se realiza la descripción del modelo cinemático mencionado.

2.1 Descripción del modelo cinemático

La Figura 2.1 representa el modelo cinemático estudiado en este trabajo. Como se puede observar, la persona se encuentra sentada sobre una silla o plataforma que gira con cierta velocidad angular.

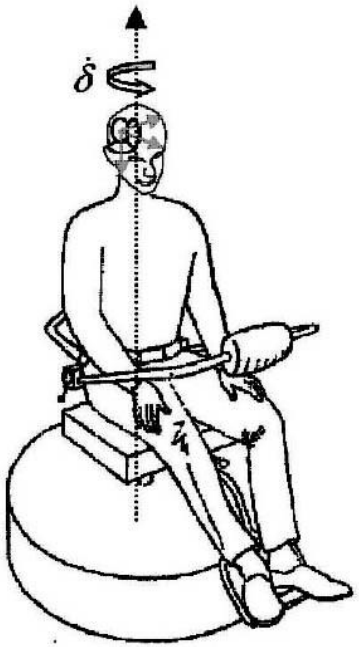


Figura 2.1 Silla donde se realiza la estimulación.

La persona sentada en la silla de rotación experimenta una estimulación del sentido del equilibrio, dicha rotación será asimilada por los canales semicirculares (en concreto por el canal lateral) de tal forma que se producirá un desplazamiento del líquido endolinfa.

En segundo lugar el modelo cinemático es debido a que, mientras la silla rota, se admite que la persona pueda girar la cabeza sobre los tres ejes X, Y, Z (alabeo, cabeceo y guiñada) (figura 2.2)

cuyos ángulos de rotación son α, β y γ respectivamente, este efecto se sumará al efecto producido por la rotación de la silla, cuyo ángulo de rotación es δ.

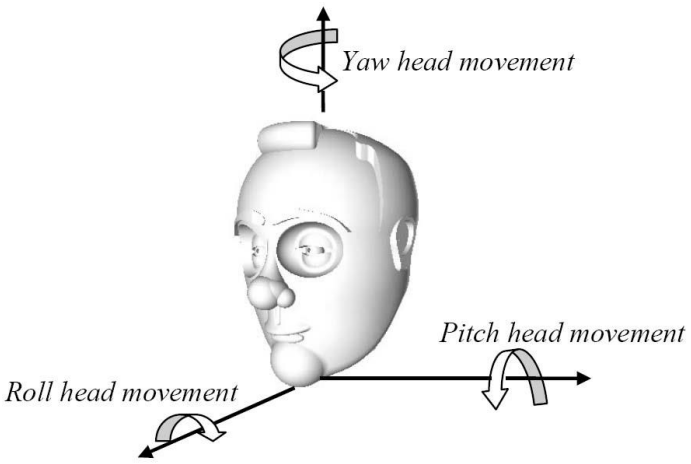


Figura 2.2 Giros de la cabeza sobre los tres ejes: alabeo (roll), cabeceo (pitch) y guiñada (yaw).

Para trabajar con este modelo cinemático se van a definir los sistemas de referencia mostrados en la Figura 2.3:

- Sistema R0 (O, X0, Y0, Z0): sistema base o terrestre.
- Sistema R1 (O, X1, Y1, Z1): sistema asociado a la silla de rotación.
- Sistema R2 (A, X2, Y2, Z2): sistema asociado a la cabeza.
- Sistema R3 (B, X3, Y3, Z3): sistema asociado a los canales semicirculares si se consideran no ortogonales.

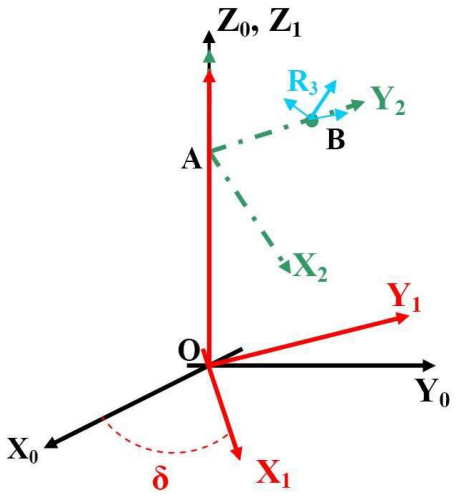


Figura 2.3 Sistema R0 (sistema terrestre), sistema R1 (sistema solidario a la silla), sistema R2 (sistema solidario a la cabeza), sistema R3 (sistema asociado a los canales semicirculares si no son considerados ortogonales).

Lo que se persigue es obtener las componentes del vector aceleración angular (que se transforma a velocidad angular previamente a ser introducido en el modelo del canal semicircular) y las componentes del vector aceleración lineal, expresándolos ambos en el sistema de referencia R0

(sistema terrestre o de referencia), conociendo el vector velocidad angular de la silla en R1 y el vector velocidad angular de la cabeza en R2.

Para obtener los vectores mencionados hay que realizar ciertas transformaciones de sistemas de referencia que se describen a continuación.

2.1.1 Vectores de velocidad angular

La velocidad angular de la silla de rotación en R1 viene dada por:

$$\bar{\omega}_1 = (0, 0, \delta') \tag{2.1}$$

Los movimientos de la cabeza quedan definidos por el siguiente vector velocidad angular en R2:

$$\bar{\omega}_2 = (\alpha', \beta', \gamma') \tag{2.2}$$

Para pasar del sistema de referencia R2 al sistema R0 se realiza la siguiente operación:

$$\begin{pmatrix} X0 \\ Y0 \\ Z0 \end{pmatrix} = M_\alpha M_\beta M_\gamma M_\delta \begin{pmatrix} X2 \\ Y2 \\ Z2 \end{pmatrix} \tag{2.3}$$

donde:

$$M_\alpha = \begin{pmatrix} 1 & 0 & 0 \\ 0 & cos\alpha & sin\alpha \\ 0 & -sin\alpha & cos\alpha \end{pmatrix} \tag{2.4}$$

$$M_\beta = \begin{pmatrix} cos\beta & 0 & -sin\beta \\ 0 & 1 & 0 \\ sin\beta & 0 & cos\beta \end{pmatrix} \tag{2.5}$$

$$M_\gamma = \begin{pmatrix} cos\gamma & sin\gamma & 0 \\ -sin\gamma & cos\gamma & 0 \\ 0 & 0 & 1 \end{pmatrix} \tag{2.6}$$

$$M_\delta = \begin{pmatrix} cos\delta & sin\delta & 0 \\ -sin\delta & cos\delta & 0 \\ 0 & 0 & 1 \end{pmatrix} \tag{2.7}$$

Se ha incluido el término M_δ para pasar del sistema R1 al sistema R0.

Entonces se calcula la velocidad angular de la cabeza en R0:

$$\bar{\omega}_{2/1} = M_\alpha M_\beta M_\gamma M_\delta \bar{\omega}_2 \tag{2.8}$$

donde el subíndice *2/1* representa la primera contribución de la componente velocidad angular de la cabeza.

De la misma manera se calcula la velocidad angular de la silla en R0 (segunda contribución):

$$\bar{\omega}_{1/0} = M_\delta \bar{\omega}_1 \tag{2.9}$$

Se obtiene el vector velocidad angular total en R0 sumando las contribuciones de la velocidad angular de la cabeza y la velocidad angular de la silla, ambos expresados en R0:

$$\bar{\omega}_{2/0} = \bar{\omega}_{2/1} + \bar{\omega}_{1/0} \tag{2.10}$$

2.1.2 Vectores de aceleración angular

Para obtener las aceleraciones angulares en el sistema R0 se opera de la misma manera que anteriormente. En primer lugar se obtiene el vector aceleración angular en R1 a partir del vector velocidad angular de la silla como:

$$\bar{\omega}'_1 = \frac{\bar{\omega}_1 - \bar{\omega}_{previo}}{t_1 - t_{previo}} \tag{2.11}$$

Donde el subíndice *previo* designa el valor del paso de simulación anterior.

Para el vector aceleración angular de la cabeza en R2 se opera igual:

$$\bar{\omega}'_2 = \frac{\bar{\omega}_2 - \bar{\omega}_{previo}}{t_1 - t_{previo}} \tag{2.12}$$

Se hacen las mismas transformaciones de sistema de referencia que en el punto anterior. Para obtener la aceleración angular de la silla en R0 se tiene:

$$\overline{\omega'}_{1/0} = M_\delta \overline{\omega'}_1 \tag{2.13}$$

Para la aceleración angular de la cabeza en R0 se calcula:

$$\overline{\omega'}_{2/1} = M_\alpha M_\beta M_\gamma M_\delta \bar{\omega}'_2 \tag{2.14}$$

Sumando las dos contribuciones se consigue el vector aceleración angular total en R0:

$$\overline{\omega'}_{2/0} = \overline{\omega'}_{2/1} + \overline{\omega'}_{1/0} \tag{2.15}$$

Como el modelo cinemático admite la posibilidad de considerar los canales semicirculares no ortogonales (sistema de referencia R3), se puede realizar una última transformación para expresar las aceleraciones angulares con esta orientación no ortogonal. Para ello las matrices de transformación serían:

$$M_{\varphi} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & cos\varphi & sin\varphi \\ 0 & -sin\varphi & cos\varphi \end{pmatrix} \tag{2.16}$$

$$M_{\theta} = \begin{pmatrix} cos\theta & 0 & -sin\theta \\ 0 & 1 & 0 \\ sin\theta & 0 & cos\theta \end{pmatrix} \tag{2.17}$$

$$M_{\psi} = \begin{pmatrix} cos\psi & sin\psi & 0 \\ -sin\psi & cos\psi & 0 \\ 0 & 0 & 1 \end{pmatrix} \tag{2.18}$$

donde ϕ, θ y ψ representan los ángulos de Euler definiendo la normal al plano de cada canal (lateral, posterior y anterior). Los valores de dichos ángulos se muestran en la Tabla 2.1.

Tabla 2.1 Ángulos de Euler en radianes que definen la perpendicular de cada canal

canal	Ángulo ψ (rad)	Ángulo θ (rad)	Ángulo ϕ (rad)
e_anterior	2,212	0,177	0
e_lateral	2,336	0	-0,274
e_posterior	0	-0,331	0,038

Se obtiene pues el vector aceleración angular antes mencionado, de la forma:

$$\overline{\omega'}_{B2/0} = M_{\varphi} M_{\theta} M_{\psi} \overline{\omega'}_{2/0} \tag{2.19}$$

2.1.3 Vectores de aceleración lineal

Los movimientos angulares también dan lugar a componentes de aceleración lineal (*A*), que contribuyen a estimular los otolitos. Los términos que componen el vector aceleración lineal en el punto B (origen del sistema R3, donde se encuentra el sistema vestibular) son:

$$\overline{A_{B2/0}} = \overline{A_{B2/1}} + \overline{\omega'}_{1/0} \times \overline{AB} + \bar{\omega}_{1/0} \times \left(\bar{\omega}_{1/0} \times \overline{AB}\right) + \overline{A_{coriolis}} \tag{2.20}$$

donde:

$$\overline{AB} = \begin{pmatrix} 0 \\ 0.03 \\ 0 \end{pmatrix} m \tag{2.21}$$

$$\overline{A_{B2/1}} = \overline{\omega'}_{2/1} \times \overline{AB} \tag{2.22}$$

Para obtener el término de la aceleración de coriolis, se necesita la velocidad lineal (*V*) del punto B, que se calcula de la forma:

$$\overline{V_{B2/1}} = \bar{\omega}_{2/1} \times \overline{AB} \tag{2.23}$$

Finalmente se calcula el término de coriolis:

$$\overline{A_{coriolis}} = 2\bar{\omega}_{1/0} \times \overline{V_{B2/1}} \tag{2.24}$$

2.2 El modelo cinemático en Matlab/Simulink

Todo el modelo cinemático descrito anteriormente se va a realizar utilizando Matlab/Simulink.

El modelo cinemático se va a realizar mediante un subsistema de Simulink que irá alojado en el modelo completo del sistema vestibular, dicho subsistema se muestra en la Figura 2.4.

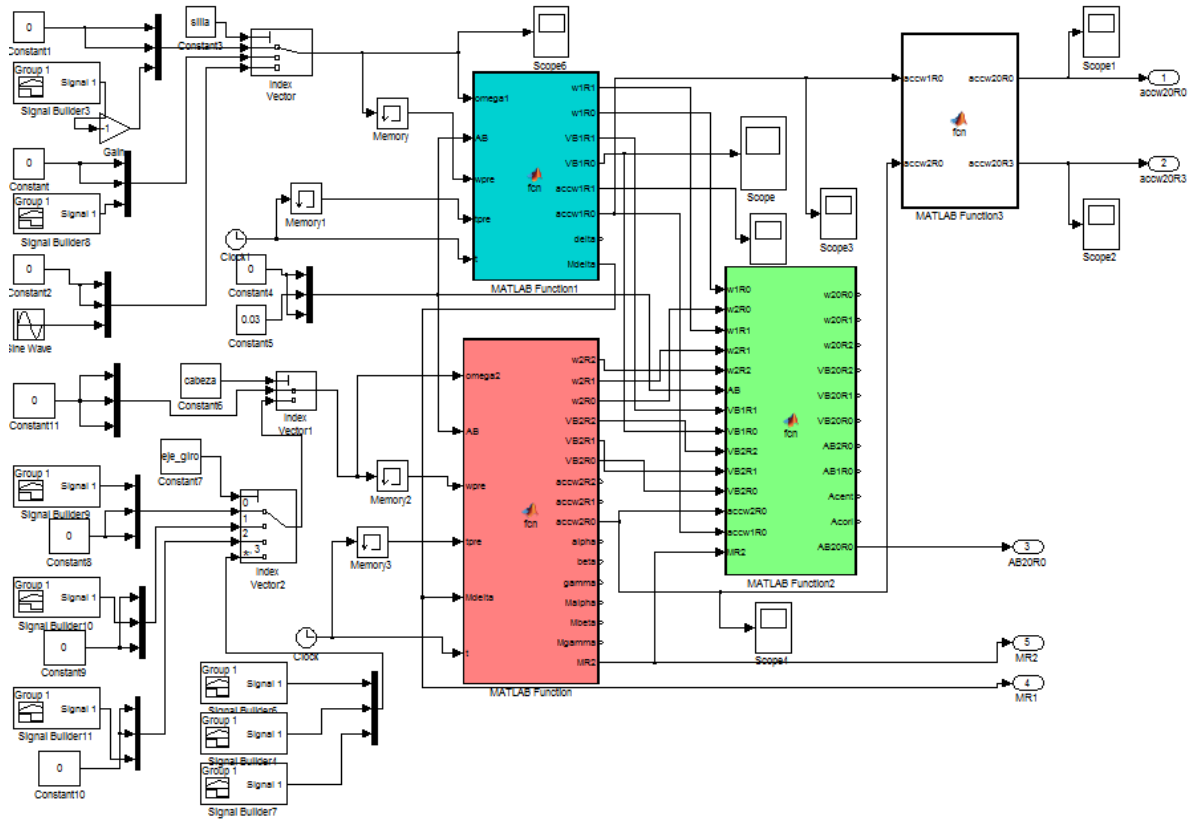


Figura 2.4 Subsistema de Simulink donde se implementa el modelo cinemático.

En primer lugar, el usuario del modelo puede seleccionar el tipo de velocidad angular que tendrá la silla de rotación, la cual puede ser constante, trapezoidal o senoidal. Para la velocidad angular de la cabeza seleccionará sobre que eje girará la cabeza (alabeo, cabeceo, guiñada) o una combinación de los tres.

Como se puede ver en la Figura 2.4 hay otros cuatro subsistemas (funciones Matlab), en cada uno se resuelven diferentes elementos del problema. El código de estas funciones Matlab se puede consultar en el correspondiente anexo de este trabajo.

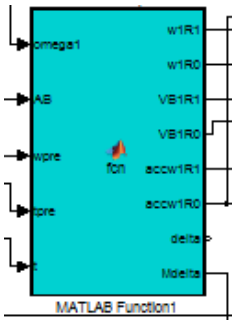


Figura 2.5 Modelo cinemático en R1.

En el subsistema azul (Figura 2.5) se obtiene la aceleración angular de la cabeza en el sistema R0 debido a la velocidad angular de la silla en el sistema R1. También se obtienen las componentes del vector velocidad lineal, necesarias para la obtención de la aceleración lineal.

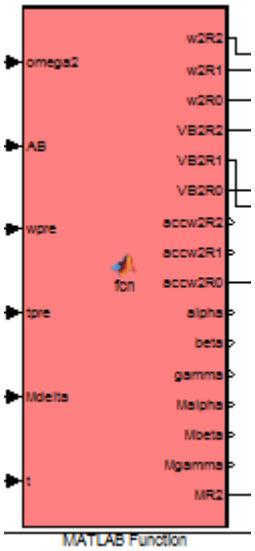


Figura 2.6 Modelo cinemático en R2.

En el subsistema rojo (Figura 2.6) se obtiene la aceleración angular de la cabeza en sistema R0 debido a la velocidad angular de la cabeza que la persona muestre en alguno de los tres ejes de rotación (alabeo, cabeceo o guiñada) en el sistema R2. Posteriormente se obtienen también las componentes del vector velocidad lineal en R0 para poder obtener la aceleración lineal.

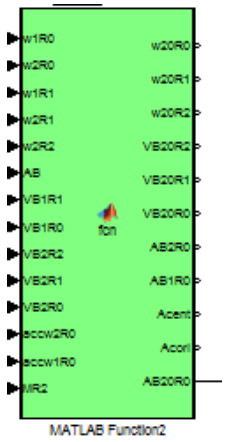


Figura 2.7 Modelo cinemático en R0.

En el subsistema verde (Figura 2.7) se obtiene el vector aceleración lineal en el sistema R0 a partir de las velocidades lineales y las aceleraciones angulares de la silla y de la cabeza. Luego se calculan la aceleración centrípeta y la aceleración de coriolis y se obtiene finalmente la aceleración lineal absoluta.

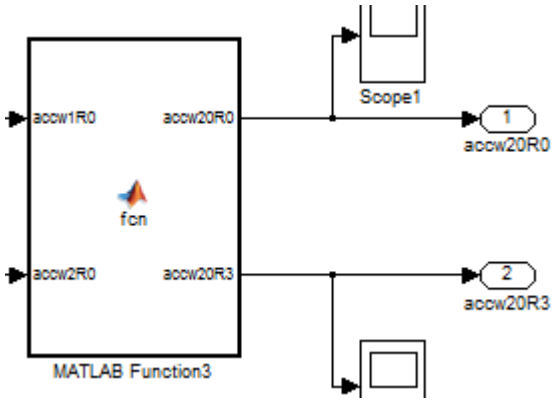


Figura 2.8 Modelo cinemático en R0/R3.

Finalmente en el subsistema blanco (Figura 2.8) se hallan los vectores de aceleración angular en el sistema R0 debido a las contribuciones de las aceleraciones angulares de la silla y la cabeza (subsistemas azul y rojo). Además del vector aceleración angular en R0, el cual considera los canales semicirculares como si fueran idealmente ortogonales, también se obtiene el vector aceleración angular, aplicando las matrices de transformación del sistema R3, que es la consideración más realista de la disposición de los canales semicirculares como si no fuesen ortogonales.

3 MODELADO Y SIMULACIÓN DEL SISTEMA VESTIBULAR EN MATLAB/SIMULINK

En este capítulo se va a realizar una deducción del modelo dinámico de los canales semicirculares y de los otolitos. Luego se van a implementar en Matlab/Simulink junto con el modelo cinemático del capítulo 2.

3.1 Modelo dinámico

Una vez que se conoce el modelo cinemático necesario para llevar a cabo la estimulación del sistema vestibular y conociendo las aceleraciones angulares y lineales de la cabeza que provocan esta estimulación, se ha de proponer un modelo dinámico en Simulink para ver el comportamiento del sistema vestibular (canales semicirculares y los otolitos).

Esto se reduce a conocer la función de transferencia del sistema de los canales semicirculares por un lado y de los otolitos por otro.

3.1.1 Modelo canales semicirculares

El primer modelo de los canales fue propuesto por W. Steinhausen (1931, 1933) [7]. Es un modelo lineal de segundo orden. Considera el canal como un toroide teniendo un radio exterior constante R, radio interior r y plantea el desplazamiento angular de la endolinfa ξ(t) sobre el centro del canal relacionado con la aceleración de la cabeza α(t) a través de la ecuación del péndulo torsional fuertemente amortiguado:

$$\ddot{\xi}(t) + \frac{\Pi}{\Theta}\dot{\xi}(t) + \frac{\Delta}{\Theta}\xi(t) = -\alpha(t) \tag{3.1}$$

donde ξ(t), ξ̇(t) y ξ̈(t) son el ángulo, la velocidad angular y la aceleración angular respectivamente del fluido; Θ es el momento de inercia de la endolinfa; Π es la viscosidad de la endolinfa; Δ es el coeficiente de resorte asociado al movimiento de la cúpula (Figura 3.1).

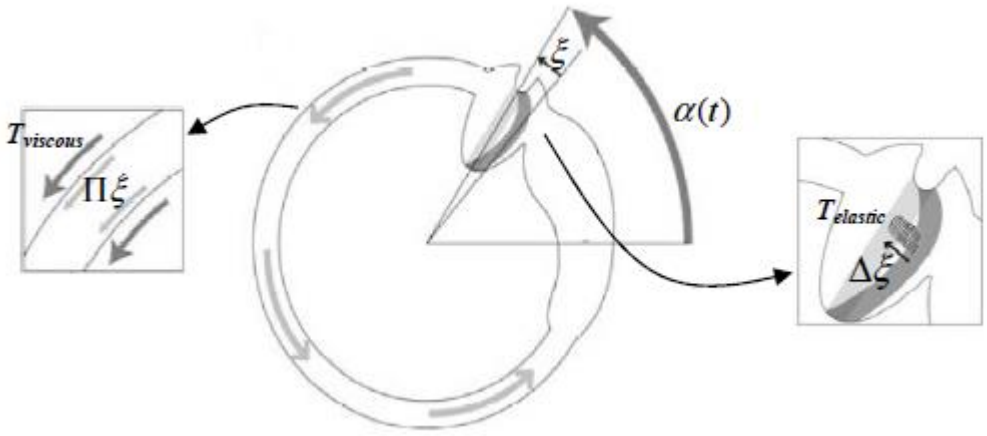


Figura 3.1 Modelo mecánico un canal semicircular.

Siendo el sistema endolinfa-cúpula altamente amortiguado, una primera simplificación de la ecuación (3.1) en el dominio de la frecuencia mediante transformada de Laplace sería:

$$\frac{\xi(s)}{\omega(s)} = \frac{(\tau_1\tau_2)}{(\tau_1 s+1)(\tau_1 s+1)} \tag{3.2}$$

donde ω es la velocidad angular de la cabeza.

Las constantes de tiempo son:

$$\frac{1}{\tau_1} = \frac{1}{\tau_2} = \frac{-\Pi \pm \sqrt{\Pi^2 - 4\Delta\Theta}}{2\Theta} \tag{3.3}$$

Un modelo más refinado que el de la ecuación (3.2) es el modelo utilizado por Oman (1987) [7] en el que a partir de la aceleración angular se obtiene el volumen desplazado por la cúpula. Además consideró el efecto de conducto elíptico de sección cruzada en un flujo de Poiseuille. Posteriormente extendió este modelo al caso en que el tamaño, la forma y la curvatura del canal cambian de forma continua a través del conducto, el utrículo y la cúpula. Obtuvo el siguiente modelo:

$$\frac{V(s)}{\alpha(s)} = \frac{-\frac{2\rho\Lambda}{K}}{\frac{\rho L}{K}\left(\frac{\bar{1}}{A}\right)s^2 + \frac{8\pi\mu L}{K}\left(\frac{\bar{S}}{A^2}\right)s + 1} \approx \frac{G}{(\tau_1 s+1)(\tau_1 s+1)} \tag{3.4}$$

donde ρ y μ son la densidad y viscosidad de la endolinfa, respectivamente. L es la distancia total del contorno, K es el coeficiente de rigidez de la ampolla, Λ es la superficie definida por el contorno central del canal y proyectada en el plano de rotación. A es el área del contorno del canal, S es un factor de forma y G es la sensibilidad del volumen desplazado de endolinfa respecto a la aceleración angular. Los valores de K y G se muestran en la Tabla 3.1.

Posteriormente se relacionó este volumen desplazado con las señales nerviosas transmitidas por los canales semicirculares conocido como “afferent firing rate” (AFR), el cual es proporcional al volumen desplazado por la cúpula. Fernández (1971) [8] obtuvo este modelo a partir del de Oman:

$$\frac{AFR(s)}{\alpha(s)} = \left[\frac{\tau_a s}{\tau_a s+1}\right]\left[\frac{\tau_i s+1}{(\tau_1 s+1)(\tau_2 s+1)}\right] \tag{3.5}$$

El primer término concuerda mejor con las bajas frecuencias. El segundo término concuerda con las desviaciones a altas frecuencias respecto del modelo péndulo torsional e implica que el sistema es sensible tanto al desplazamiento de la cúpula como a la velocidad de desplazamiento.

Los valores de las cuatro constantes de tiempo son se muestran en la Tabla 3.1.

Damiano y Rabbitt (1996) [7] obtuvieron el modelo para un canal semicircular individual ignorando la comunicación de fluido con los dos canales asociados. Consiguieron la siguiente ecuación del comportamiento mecánico:

$$m\frac{d^2Q}{dt^2} + c\frac{dQ}{dt} + kQ = f \tag{3.6}$$

donde Q es el volumen desplazado de endolinfa, m es la masa del fluido contenido en el canal, c describe el efecto viscoso amortiguado en el conducto, k es la rigidez de la cúpula y f es una fuerza inercial.

Pasando al dominio de Laplace y realizando los correspondientes cambios de unidades, se tiene la siguiente función de transferencia [7]:

$$T_{ssc}(s) = \frac{Q(s)}{\omega(s)} = \frac{\left(\frac{d}{m}\right)s}{\left(s+\frac{1}{\tau_1}\right)\left(s+\frac{1}{\tau_2}\right)} \tag{3.7}$$

donde d es un coeficiente inercial de fuerza. Esta función de transferencia, relaciona el volumen desplazado por la cúpula en cm^3 con la velocidad angular de la cabeza en rad/s. Los valores de m, d y de las constantes de tiempo se muestran en la Tabla 3.1.

Este último modelo es el que se va a utilizar en este trabajo.

La Tabla 3.1 muestra los valores de los parámetros de las ecuaciones (3.4), (3.5) y (3.7).

Tabla 3.1 Parámetros de los modelos dinámicos

Parámetro	Valor	Descripción
K	6,8 GPa/m^3	coeficiente de rigidez de la ampolla
G	1,85x10^{-11} m^3s^2	sensibilidad del volumen desplazado de endolinfa respecto a la aceleración angular
τ_a	80 s	cte. de tiempo (3.5)
τ_i	0,049 s	cte. de tiempo (3.5)
τ_1	5,7 s	cte. de tiempo (3.5)
τ_2	0,003 s	cte. de tiempo (3.5)
m	1070 g/cm^4	masa del fluido contenido en el canal
d	0,76 g/cm	coeficiente inercial de fuerza
τ_1	13,2 s	cte. de tiempo (3.7)
τ_2	0,006 s	cte. de tiempo (3.7)

3.1.2 Modelo otolitos

El primer modelo de otolitos fue planteado por Meiry (1966) [7], con el que se obtenía la velocidad a partir de un estímulo de velocidad.

Más tarde Young and Meiry (1968) [7] propusieron un modelo modificado que relacionaba fuerza gravito-inercial con estímulos de fuerza específicos.

Años más tarde, Zacharias (1977) [7] planteó un modelo para representar la dinámica de los otolitos, el cual es similar al modelo del péndulo torsional de los canales semicirculares. Ormsby (1974) y Grant (1986, 1987, 1990, 1994) [7] propusieron versiones refinadas. Consideraban la capa gelatinosa soportando la otoconia como un sólido elástico.

Grant y Cotton (1990) [7] propusieron la introducción de una amortiguación adicional considerando el modelo mecánico de la capa gelatinosa viscoelástica. Consideraron los otolitos como un amortiguador de masa de segundo orden, donde la capa otoconia era modelada como un sólido rígido, la capa gel era modelada como un material isotrópico viscoelástico y la endolinfa se modelaba como un fluido Newtoniano con viscosidad uniforme (Figura 3.2).

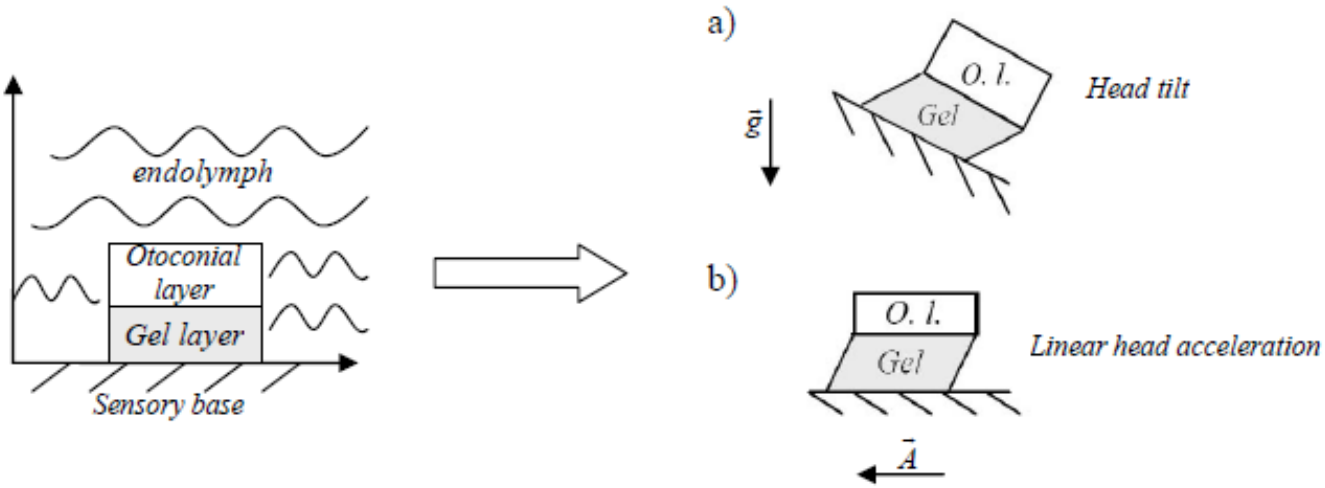


Figura 3.2 Diagrama esquemático del funcionamiento del órgano otolito. a) Estímulo inclinación de la cabeza. b)Estímulo inercial.

Finalmente, aplicando la segunda ley de Newton en el plano de la capa otoconia ellos obtuvieron la función de transferencia:

$$\frac{X(s)}{F(s)} = [1 - \frac{\rho_e}{\rho_0}][\frac{\tau_{1oto}\tau_{2oto}}{(\tau_{1oto}s + 1)(\tau_{2oto}s + 1)}] \quad (3.8)$$

Donde x es el desplazamiento relativo de la capa otoconia respecto a la cabeza, ρe es la densidad de la endolinfa y ρ_0 es la densidad de la membrana otoconia. Por otro lado τ_{1oto} y τ_{2oto} son las constantes de tiempo larga y corta respectivamente, que caracterizan la respuesta dinámica temporal macromecánica de la capa otoconia.

Fernandez y Goldberg (1976) [8] llevaron a cabo la más completa investigación y plantearon el modelo que contempla la respuesta de la transmisión de señales nerviosas de los otolitos ante un estímulo de aceleración lineal. Estudiaron la descarga en las neuronas otolitos de un mono ardilla en respuesta a variaciones de fuerza sinusoidal. Midieron descargas neuronales regulares e irregulares. Caracterizaron la respuesta en frecuencia con una función de transferencia de la forma:

$$H(s) = K_{oto}\frac{1+k_a\tau_a s}{1+\tau_a s}\cdot\frac{1+k_v(\tau_v s)^{k_v}}{1+\tau_M s} = K_{oto}H_a(s)\frac{H_v(s)}{H_M(s)} \tag{3.9}$$

El término $H_a(s)$ es operador de adaptación que contribuye a liderar las bajas frecuencias e incrementa la ganancia desde estática o cero hasta 0.006Hz. El término $H_v(s)$ es un operador de sensibilidad-velocidad con un exponente fraccional (kv < 1). El último término, $H_M(s)$, es un operador de primer orden que el cual Fernandez y Goldberg se dieron cuenta que podía reflejar el movimiento del otolito. Este término de retardo cuenta para los retardos de fase para altas frecuencias observados, en unidades regulares y para frecuencias de avance en unidades irregulares. El término K_{oto} define la sensibilidad estática en términos del afferent firing rate por unidad de aceleración, en unidades de impulsos por segundo por g (ips/g).

Fernandez y Goldberg [8] estimaron los parámetros de la función de transferencia (3.9). La mediana de los parámetros para ambas, unidades regulares e irregulares, para τ_v=40 segundos, se muestran en la Tabla 3.2.

Tabla 3.2 Parámetros de la ecuación (3.9)

Parámetro	Unidades regulares	Unidades irregulares
k_v	0,188	0,440
k_a	1,12	1,90
τ_a	69 s	101 s
τ_M	16 ms	9 ms
k_{oto}	25,6 ips/g	20,5 ips/g

Posteriormente Hosman (1996) [8] se dio cuenta de que el modelo anterior no tenía una fácil implementación debido al exponente fraccional y por tanto propuso una simplificación al modelo que sería refinada por Telban y Cardullo (2005) [8], dando lugar a una función de transferencia que relaciona la transmisión de señales nerviosas de la dinámica del otolito (afferent firing rate AFR) con una fuerza gravito-inercial de entrada al sistema. La función de transferencia mencionada es de la forma:

$$\frac{AFR(s)}{F(s)} = 33{,}3\frac{10s+1}{(5s+1)(0{,}016s+1)} \tag{3.10}$$

Siendo F(s) la transformada de la fuerza gravito-inercial o la aceleración lineal en m/s^2. Esta aceleración será la que se ha obtenido del modelo cinemático del capítulo 2 además de la

aceleración lineal que se considere que tiene la cabeza de la persona debido al movimiento lineal, si es que se considera.

Este último modelo de los otolitos será el que se utilice en este trabajo.

3.2 Implementación en matlab/simulink

Todo lo visto en los capítulos 2 y 3 de este trabajo se ha implementado con el programa Matlab. Se ha construido un modelo en Simulink que engloba tanto el modelo dinámico de los canales semicirculares y los otolitos, como el modelo cinemático.

Se obtiene así un modelo completo del sistema vestibular que se muestra en la Figura 3.3.

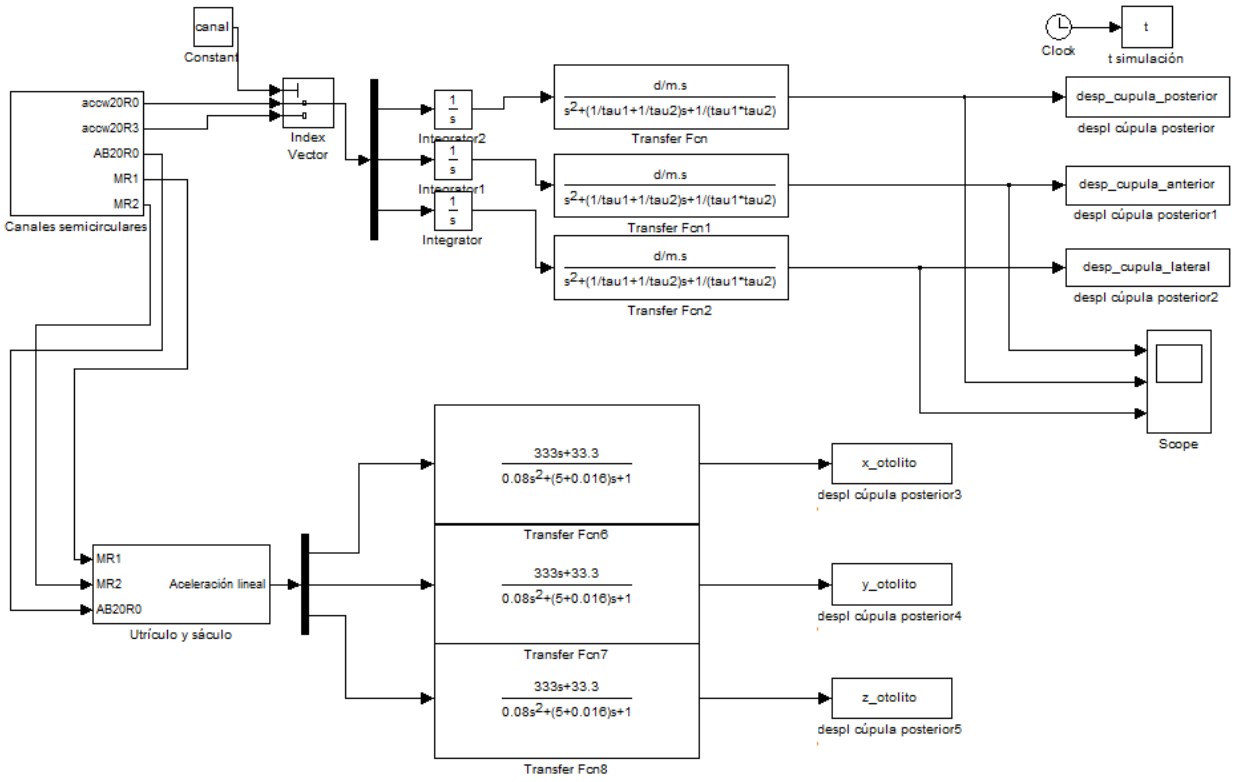


Figura 3.3 Modelo completo del sistema vestibular en Simulink.

Además se ha utilizado un archivo m-file para asignar todos los valores de las constantes y demás parámetros del sistema, así como los plots para ver las representaciones gráficas de la respuesta del sistema (el código de este archivo se puede consultar en el correspondiente anexo de este trabajo).

Conforme a la Figura 3.3, el subsistema *canales semicirculares* es donde se realiza el modelo cinemático como se ha mostrado en el punto 2.2 del presente trabajo. Las componentes del vector aceleración angular resultante de este subsistema, ya sean expresadas en el sistema R0 (consideración de canales semicirculares ortogonales) o en el sistema R3 (consideración de los canales semicirculares no ortogonales) se integran para obtener el vector velocidad angular y posteriormente cada componente de este vector se introduce en la función de transferencia expresada por la ecuación 3.8. Finalmente se obtiene el volumen de endolinfa desplazado por la cúpula en los tres canales semicirculares (posterior, anterior y lateral).

3.2.1 Subsistema de los otolitos

Por otro lado, como se ve en la Figura 3.3, se encuentra el subsistema *Utrículo y sáculo*, donde se obtiene el vector aceleración lineal que se introducirá en la función de transferencia de los otolitos. El interior de dicho subsistema se puede ver en la Figura 3.4.

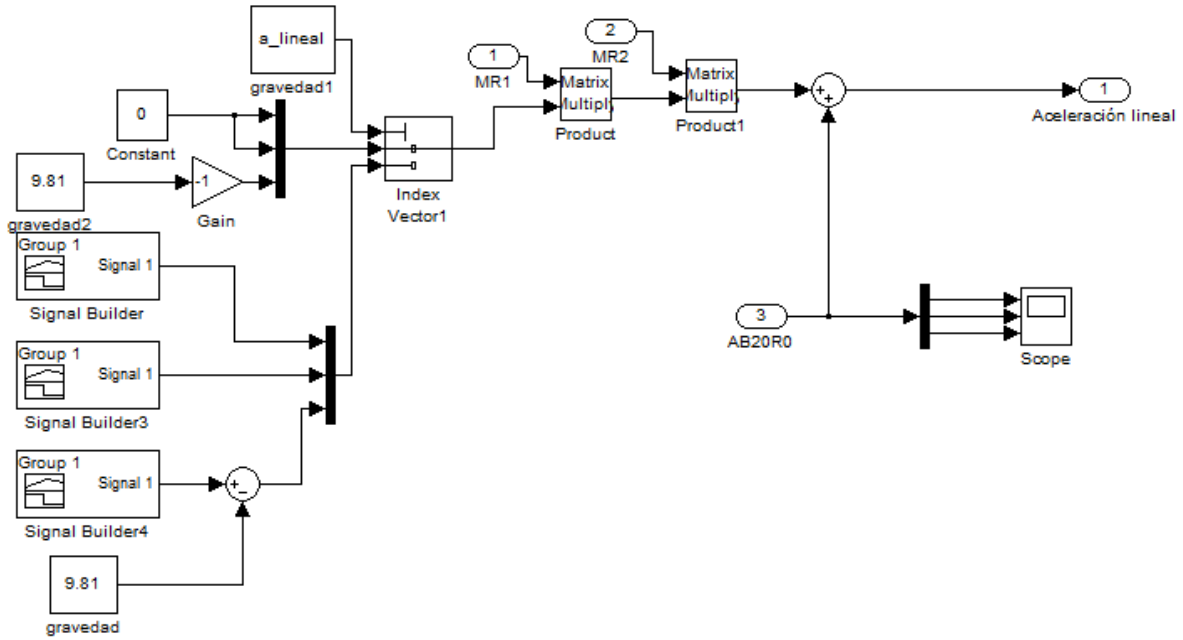


Figura 3.4 Subsistema Utrículo y sáculo.

Conforme a la Figura 3.4, el usuario puede elegir si la persona experimenta un movimiento de traslación, es decir, al vector aceleración lineal obtenido del movimiento angular (el que se ha calculado en el subsistema *canales semicirculares*) se le sumará otro vector de aceleración lineal, o bien, que esté estática traslacionalmente, por lo que solamente se tendrá la contribución de aceleración lineal debida al movimiento angular de la cabeza.

Como se ha comentado este subsistema considera que la persona pueda experimentar un movimiento de traslación (vector aceleración lineal) al que hay que sumar el vector aceleración lineal AB20R0, procedente del subsistema *canales semicirculares* y que corresponde a la aceleración lineal debida al movimiento angular.

El vector aceleración lineal debido al posible movimiento de traslación se pasa al sistema R0 mediante las matrices de transformación explicadas en el capítulo 2. Posteriormente se suma al vector de aceleración lineal procedente del subsistema *canales semicirculares.* Por último se introducen cada una de las componentes del vector de aceleración lineal en la función de trasferencia de los otolitos definida por la ecuación (3.10). Finalmente se obtiene el AFR (afferent firing rate), que es el estímulo en los ejes X, Y, Z procedente del sistema vestibular, que reciben las neuronas.

3.3 Simulaciones

En este apartado se van a realizar varias simulaciones del modelo en Simulink visto en la Figura 3.3 para las diferentes opciones de estímulo que se tienen.

3.3.1 Simulación para velocidad constante de la silla

Se ha realizado una simulación en la que solo se mueve la silla, para una función velocidad angular constante de la silla (Figura 3.5).

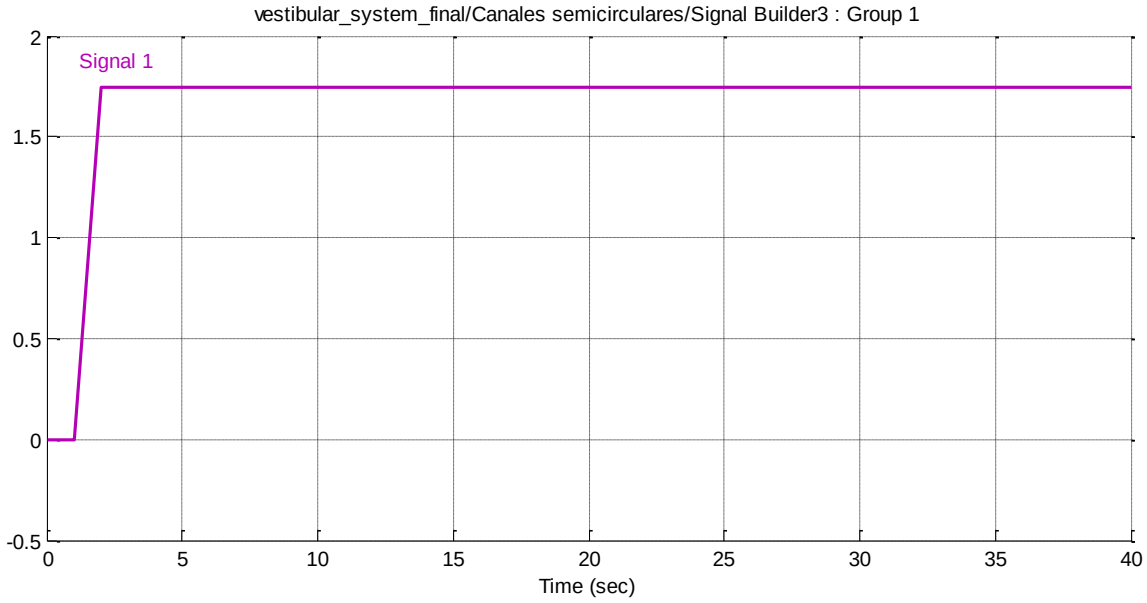


Figura 3.5 Velocidad angular constante de la silla en rad/s.

En esta primera simulación no se considera que la persona mueva la cabeza, por tanto, se obtienen los volúmenes desplazados en los canales semicirculares considerándolos ortogonales mostrados en la Figura 3.6.

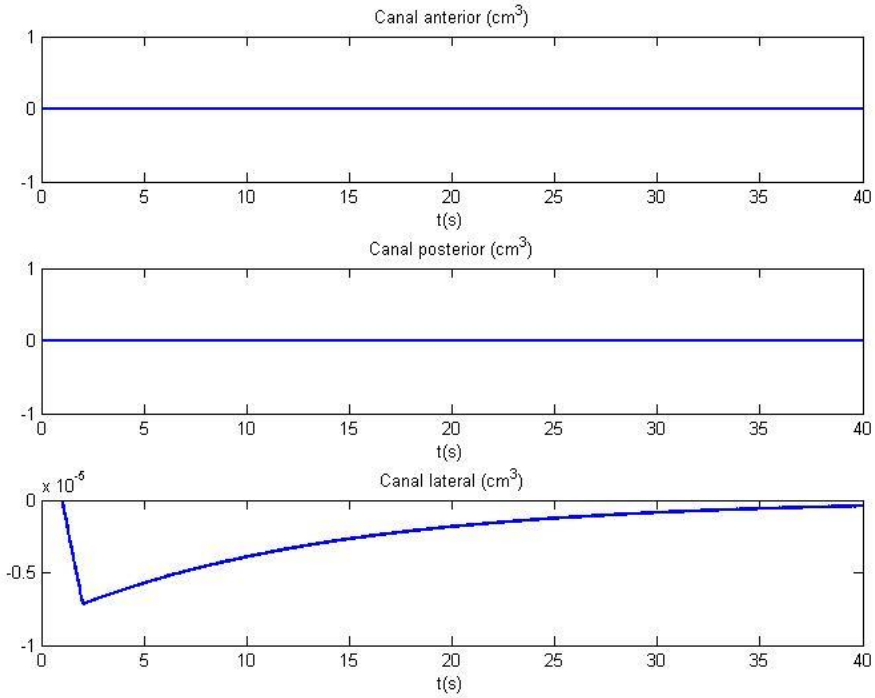


Figura 3.6 Volumen desplazado canales semicirculares ortogonales.

Si se consideran no ortogonales (aplicando el cambio al sistema R3) se obtiene la Figura 3.7.

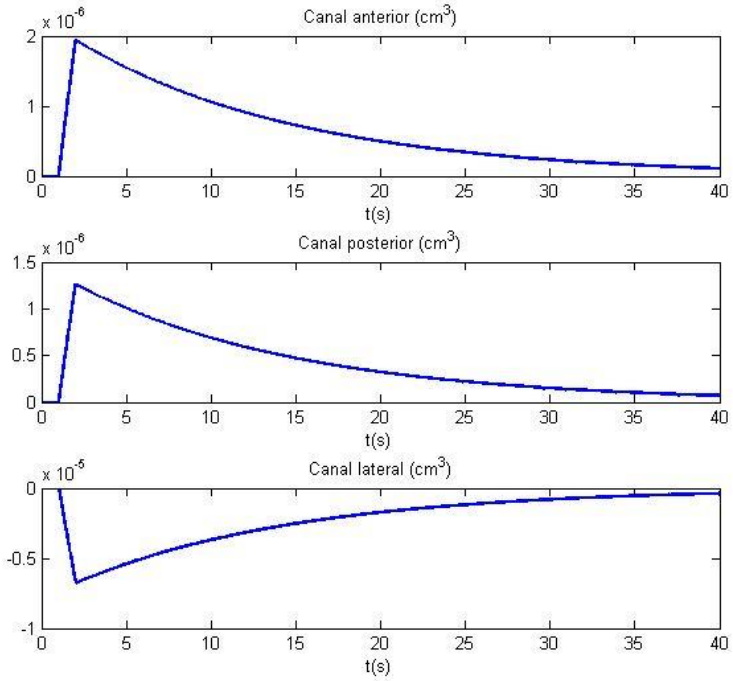


Figura 3.7 Volumen desplazado canales semicirculares no ortogonales.

A raíz de los resultados se puede apreciar que en el caso de la consideración ortogonal (Figura 3.6), es claro que solo se estimula el canal lateral dado que la silla estimula el giro sobre el eje Z de la cabeza (guiñada). Si se consideran no ortogonales (Figura 3.7) el canal lateral sufre la mayor estimulación. Sin embargo tanto el canal anterior como el posterior sufren también un desplazamiento de volumen, aunque menor que el lateral y en diferente sentido.

En cuanto a los otolitos, se obtiene la respuesta mostrada en la Figura 3.8, sin considerar movimiento traslacional.

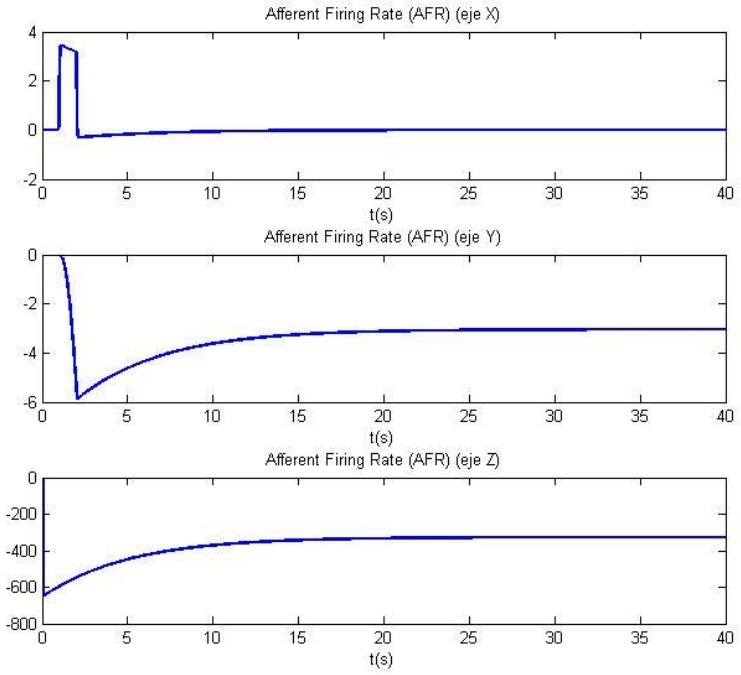


Figura 3.8 AFR en los ejes X, Y, Z de los otolitos.

Como se puede ver (Figura 3.8) la mayor estimulación en términos de AFR (afferent firing rate) es en el eje Z debido a la aceleración de la gravedad, siendo en el eje X y en el eje Y despreciable comparando el orden de magnitud.

Seguidamente se va a considerar que hay movimiento traslacional con un vector de aceleración lineal cuyas componentes en los ejes X, Y, Z son como muestra la Figura 3.9.

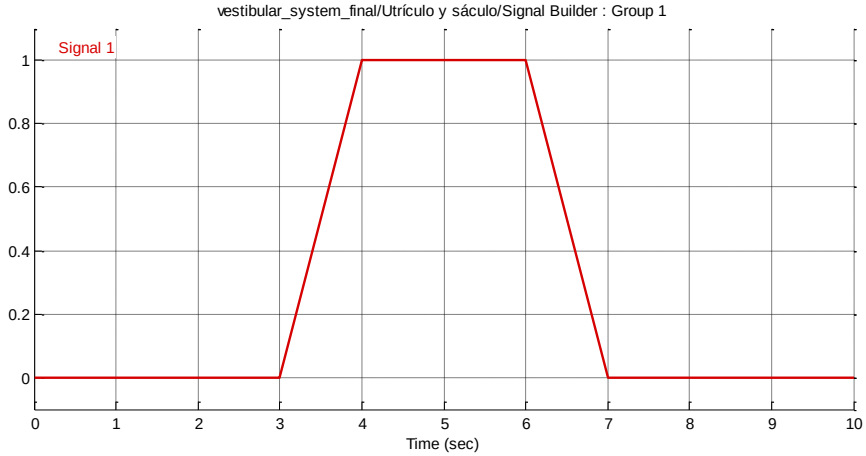


Figura 3.9 Componente de aceleración lineal en cada eje en m/s2.

De esta forma se consigue la respuesta de los otolitos de la Figura 3.10.

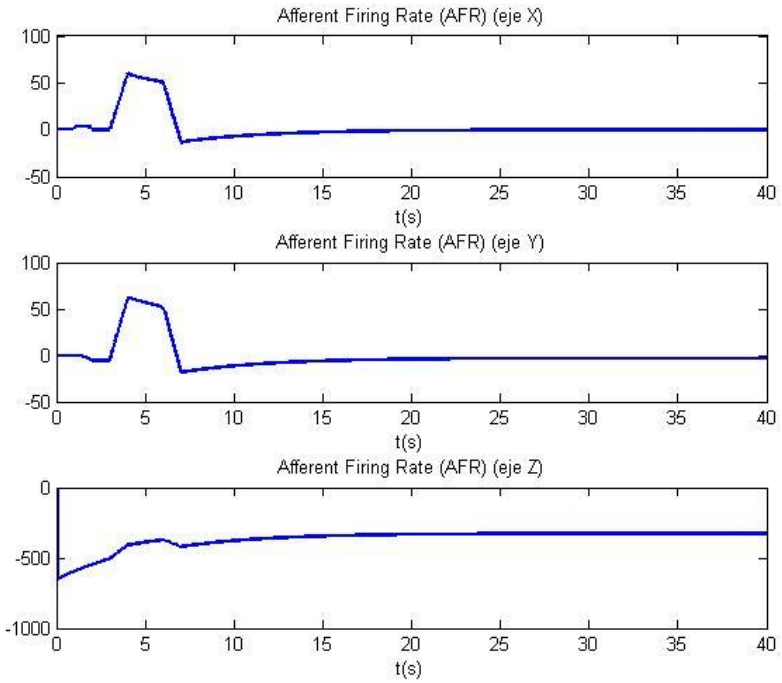


Figura 3.10 AFR en los ejes X, Y, Z de los otolitos.

Aunque la mayor estimulación sigue siendo en el eje Z debido a la gravedad (Figura 3.10), se puede apreciar que ahora la estimulación en los ejes X e Y es mucho mayor dado que están experimentando una aceleración de 1 m/s^2 ambos ejes.

Si se considera ahora que la cabeza experimenta diferentes movimientos en los diferentes ejes de giro en los instantes t=10s y t=25s como se muestra en la Figura 3.11 y 3.12.

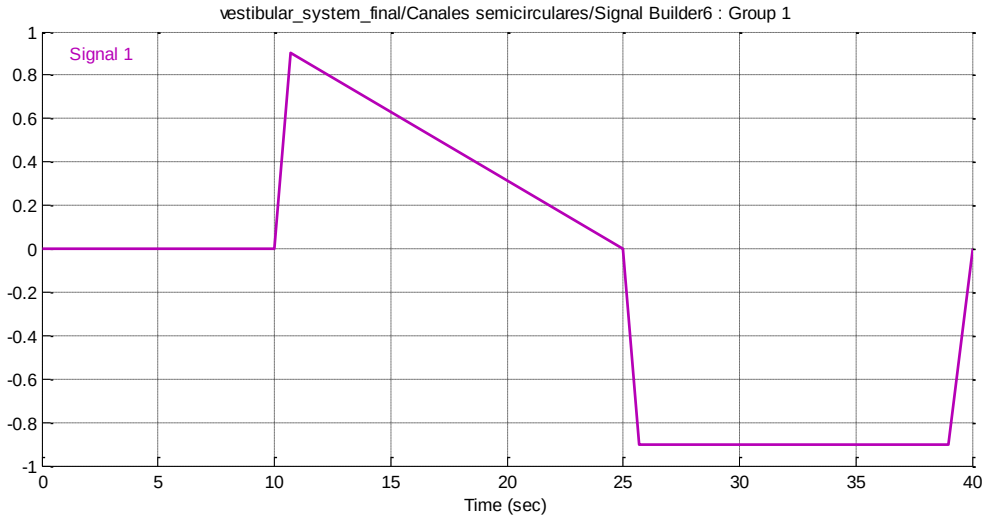


Figura 3.11 Velocidad angular de la cabeza en el eje X (alabeo) y en el eje Z (guiñada) en rad/s.

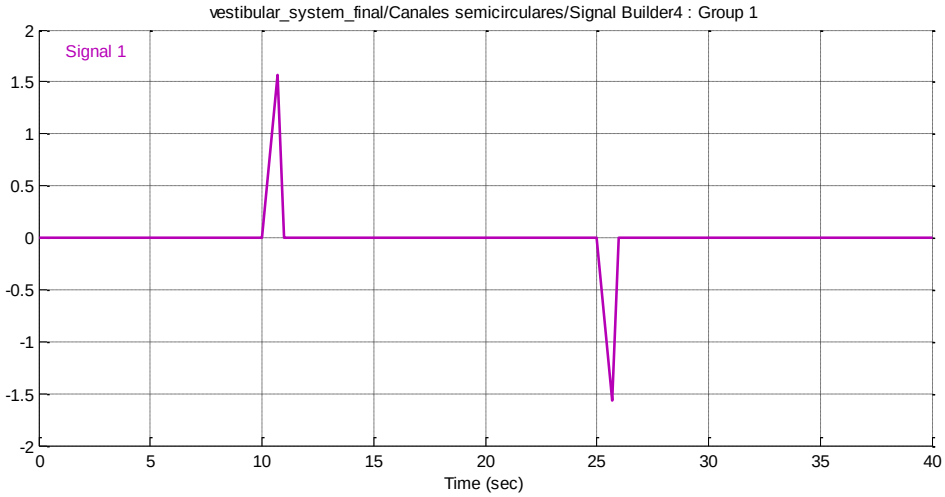


Figura 3.12 Velocidad angular de la cabeza en el eje Y (cabeceo) en rad/s.

Considerando los canales semicirculares ortogonales se obtienen los volúmenes desplazados de la Figura 3.13.

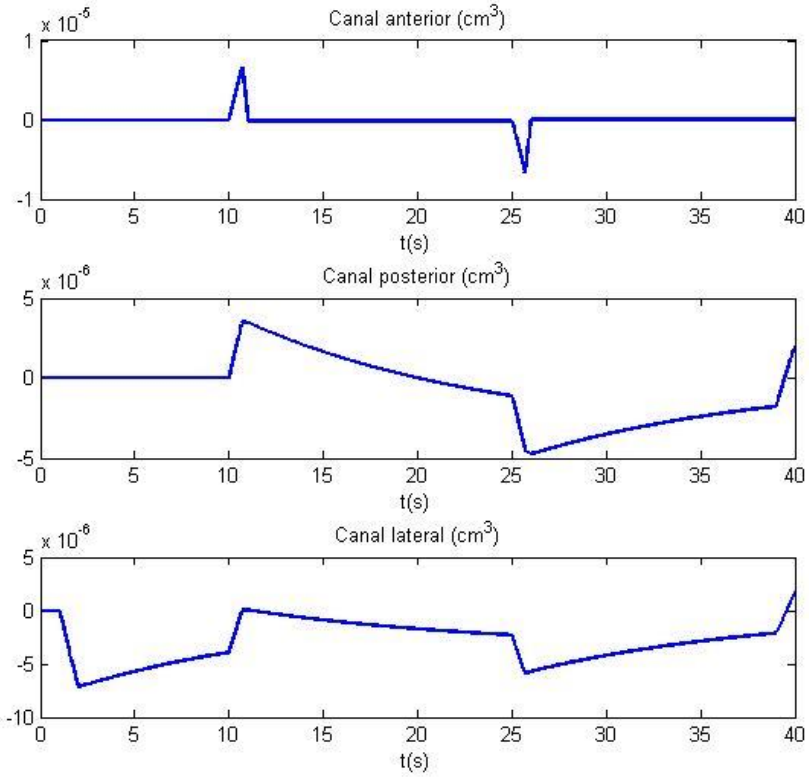


Figura 3.13 Volumen desplazado canales semicirculares ortogonales.

Si se consideran no ortogonales (aplicando el cambio al sistema R3) se obtiene la Figura 3.14.

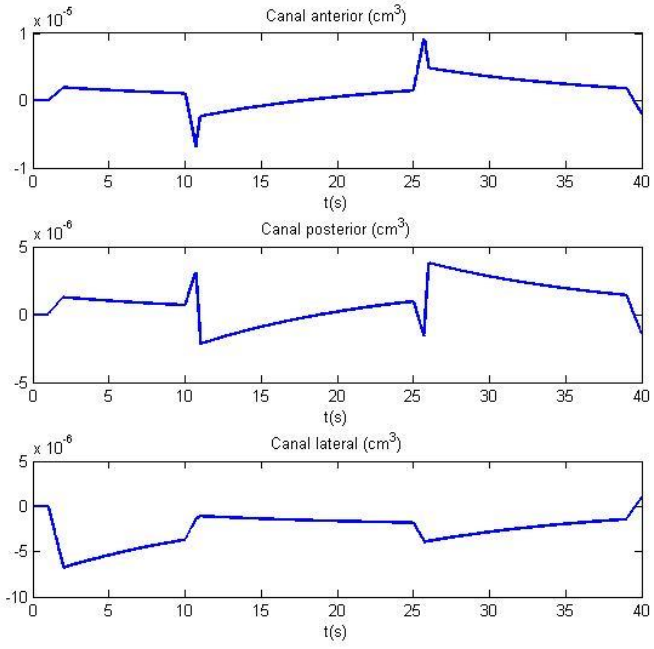


Figura 3.14 Volumen desplazado canales semicirculares no ortogonales.

De los resultados se deduce que en el caso de la consideración de canales ortogonales (Figura 3.13), cada canal semicircular se estimula con la componente del vector aceleración angular que corresponde a su respectivo eje ortogonal (anterior con la componente Y; posterior con la componente X; lateral con la componente Z). Además como se aprecia en la Figura 3.13 el canal lateral sufre primero el desplazamiento de volumen debido a la aceleración angular que produce la silla en la cabeza y después la del giro en el eje Z (guiñada) de la propia cabeza de la persona en los instantes t=10s y t =25s.

En la consideración de canales no ortogonales, se aprecia de la Figura 3.14, que se produce un volumen desplazado en los tres canales con el movimiento de la silla rotatoria, que evidentemente es mayor en el canal lateral como se había comentado en la simulación previa. No obstante en el instante t=10s y en el instante t=25s, los canales anterior y posterior comienzan a experimentar un desplazamiento de volumen mayor que se debe obviamente a la componente del vector aceleración angular de la cabeza que coincide con el eje ortogonal del respectivo canal semicircular. El canal lateral sufre un desplazamiento de volumen muy similar al del caso de canales ortogonales.

En cuanto a los otolitos se obtiene la respuesta de la Figura 3.15, sin considerar movimiento traslacional.

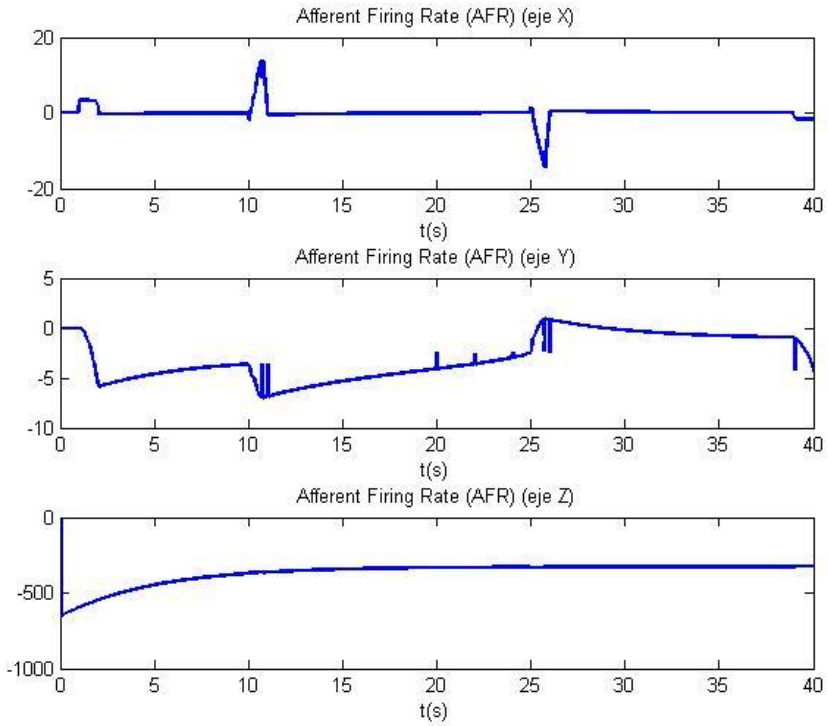


Figura 3.15 AFR en los ejes X, Y, Z de los otolitos.

Se puede ver (Figura 3.15) que los otolitos se ven estimulados por los diferentes movimientos de la cabeza aunque el AFR es despreciable en los ejes X e Y si los comparamos con el AFR en el eje Z donde actúa la gravedad.

Si se introducen las componentes del vector aceleración lineal de la Figura 3.9 se obtiene la Figura 3.16.

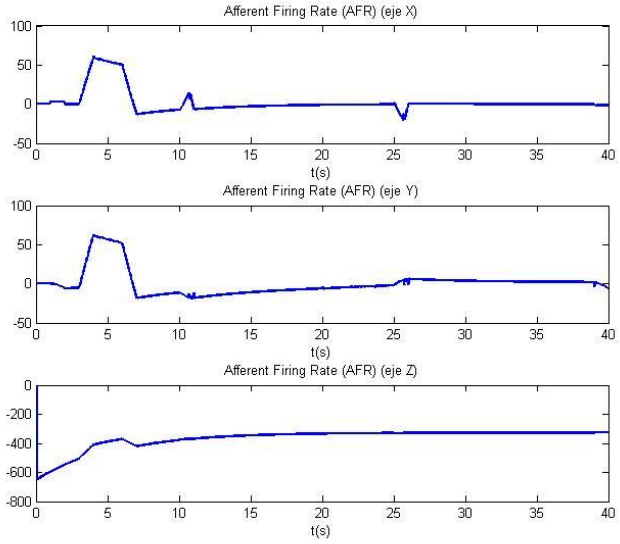


Figura 3.16 AFR en los ejes X, Y, Z de los otolitos.

Es claro que las componentes de aceleración lineal del Figura 3.9 consiguen una mayor estimulación de los otolitos (Figura 3.16) que las componentes de aceleración lineal procedentes del movimiento angular de la cabeza. En cualquier caso la mayor estimulación sigue siendo en el eje Z debido a la aceleración de la gravedad, como en los casos anteriores.

3.3.2 Simulación para velocidad trapezoidal de la silla

En esta simulación se prestará atención únicamente al comportamiento de los canales semicirculares para una función velocidad angular de la silla trapezoidal, mostrada en la Figura 3.17.

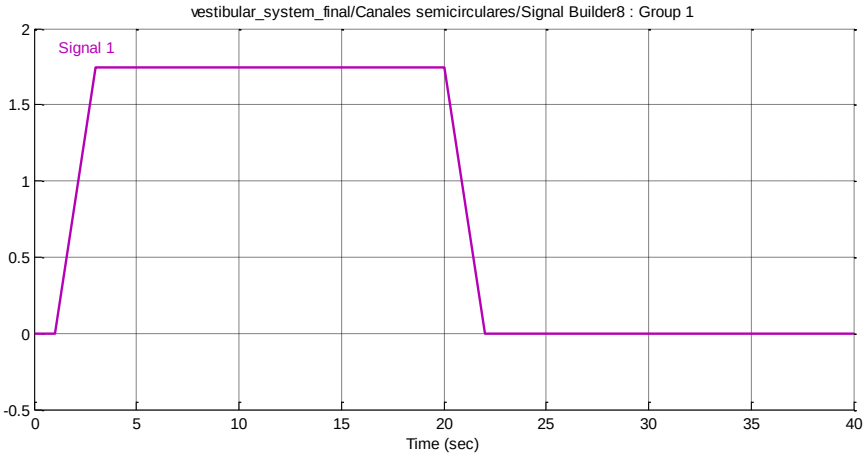


Figura 3.17 Velocidad angular trapezoidal de la silla en rad/s.

En primer lugar no se considera que la persona mueva la cabeza, por tanto, se obtienen los volúmenes desplazados de la Figura 3.18 en los canales semicirculares, considerándolos ortogonales.

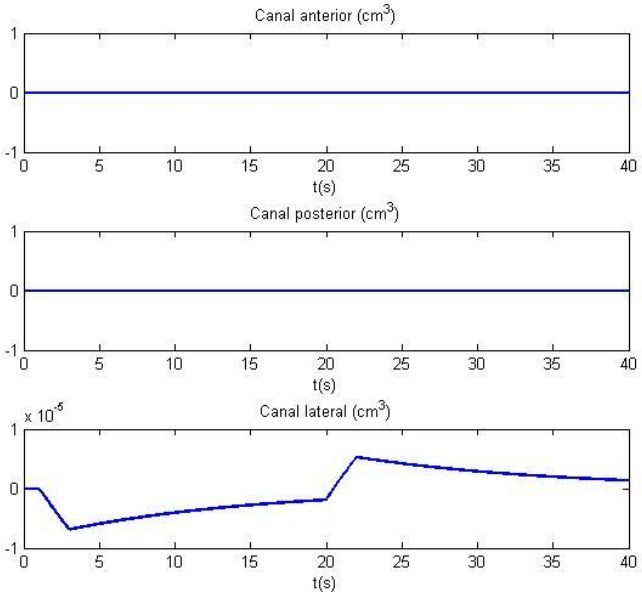


Figura 3.18 Volumen desplazado canales semicirculares ortogonales.

Se comprueba que el canal estimulado es el lateral (Figura 3.18), en el instante 1 segundo comienza el desplazamiento debido a la velocidad de la silla, luego cuando la silla decelera en el instante 20 segundos y pierde velocidad el volumen se desplaza en el otro sentido con el mismo valor a la aceleración inicial.

Si se consideran no ortogonales (aplicando el cambio al sistema R3) se obtiene la Figura 3.19.

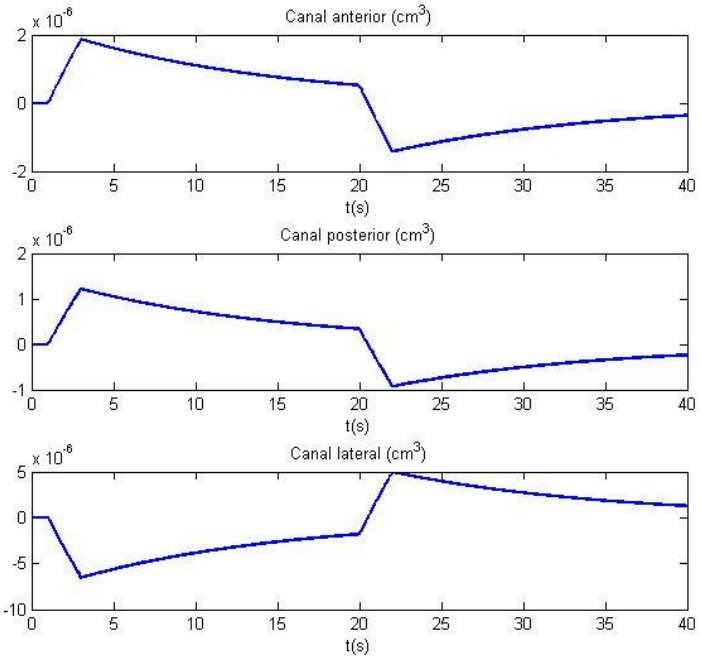


Figura 3.19 Volumen desplazado canales semicirculares no ortogonales.

Si se consideran no ortogonales (Figura 3.19) el canal lateral sufre la mayor estimulación, sin embargo tanto el canal anterior como el posterior sufren también un desplazamiento de volumen, aunque menor que el lateral y en diferente sentido, como ocurría para la velocidad constante de la silla.

Los tres canales primero se estimulan en un sentido debido a la aceleración inicial (velocidad creciente de la silla) luego el volumen desplazado cambia de sentido debido a la deceleración (velocidad decreciente de la silla).

Al igual que antes, si se considera ahora que la cabeza experimenta diferentes movimientos en los diferentes ejes de giro en los instantes t=10s y t=25s como se muestra en las figuras 4.9 y 4.10 además de considerar los canales semicirculares ortogonales se obtienen los volúmenes desplazados mostrados en la Figura 3.20.

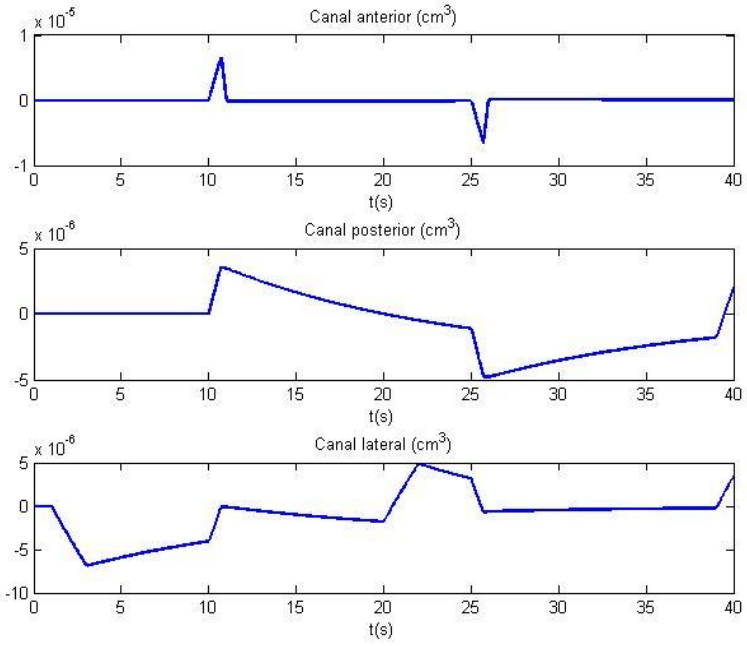


Figura 3.20 Volumen desplazado canales semicirculares ortogonales.

Se obtienen los mismos volúmenes desplazados en el canal anterior y posterior que en el caso de velocidad constante de la silla (Figura 3.20), sin embargo el canal lateral experimenta primero una desplazamiento negativo del volumen de endolinfa y sufre dos desplazamientos positivos seguidos debido al giro de la cabeza en sentido contrario a movimiento de la silla en primer lugar y a la desaceleración de la silla en segundo lugar.

Si se consideran no ortogonales (aplicando el cambio al sistema R3) se obtiene la Figura 3.21.

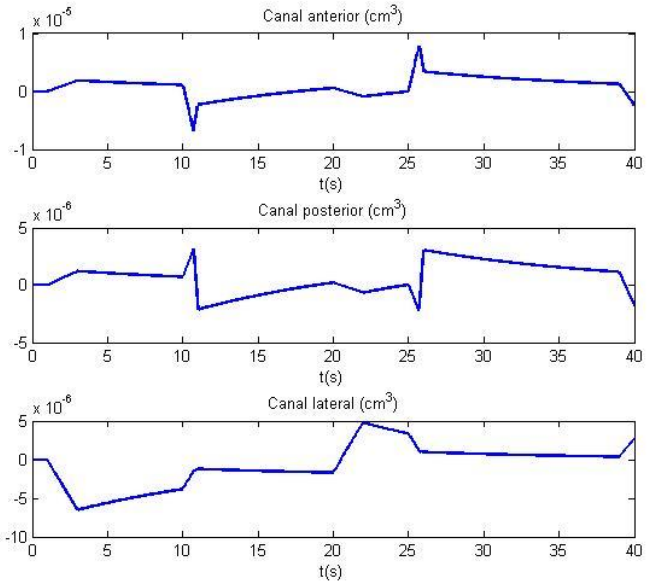


Figura 3.21 Volumen desplazado canales semicirculares no ortogonales.

Se puede apreciar que se obtiene una respuesta muy similar al caso de la velocidad angular constante de la silla en los canales anterior y posterior (Figura 3.21), salvo por el hecho de que en el instante 20 segundos hay un pequeño desplazamiento de volumen debido a la deceleración de la velocidad angular de la silla. El canal lateral sufre la misma respuesta que si se consideran ortogonales.

3.3.3 Simulación para velocidad senoidal de la silla

En esta simulación, de nuevo, se prestará atención únicamente al comportamiento de los canales semicirculares para una función velocidad angular de la silla senoidal, mostrada en la Figura 3.22.

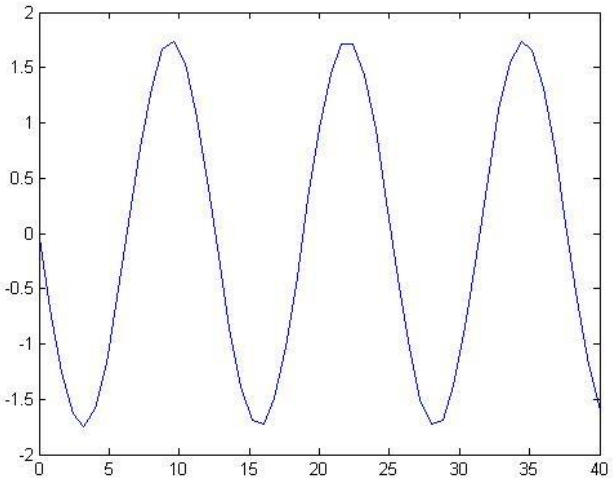


Figura 3.22 Velocidad angular senoidal de la silla en rad/s.

En primer lugar no se considera que la persona mueva la cabeza, por tanto, se obtienen los volúmenes desplazados de la Figura 3.23 en los canales semicirculares, considerándolos ortogonales.

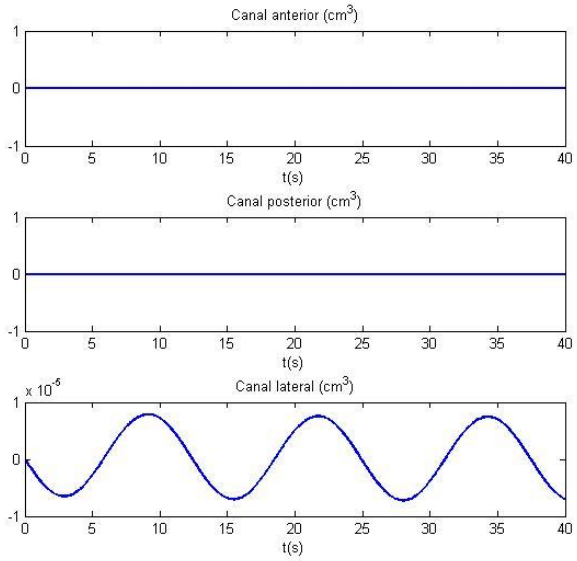


Figura 3.23 Volumen desplazado canales semicirculares ortogonales.

Se comprueba que el canal estimulado es el lateral (Figura 3.23). La respuesta es diferente al de los casos anteriores, puesto que en esta ocasión la velocidad angular de la silla es senoidal. Se aprecia que la respuesta del canal lateral tiene la misma frecuencia que la velocidad angular de la silla. El volumen desplazado va adoptando la forma del seno y cambiando entre sentido positivo y negativo, según la velocidad angular de la silla pasa de positiva a negativa.

Si se consideran no ortogonales (aplicando el cambio al sistema R3) se obtiene la Figura 3.24.

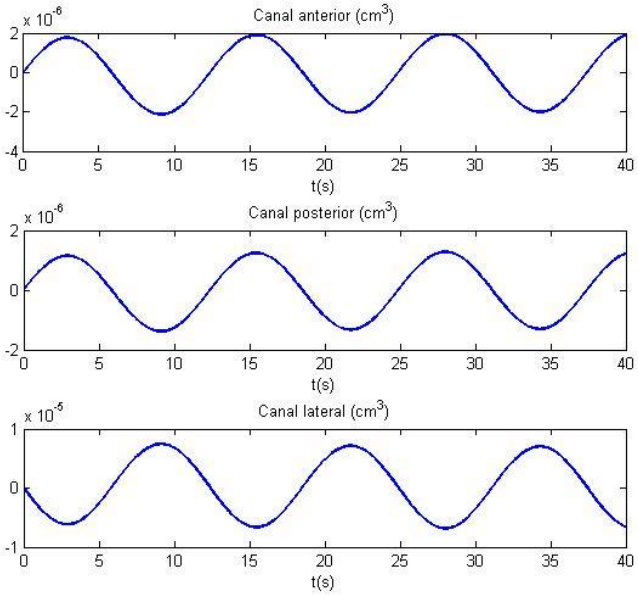


Figura 3.24 Volumen desplazado canales semicirculares no ortogonales.

Como en casos anteriores el sentido del volumen desplazado en los canales anterior y posterior es el contrario al del canal lateral (Figura 3.24). Los valores del volumen desplazado siguen siendo mayores en el canal lateral y los órdenes de magnitud son iguales a los de las simulaciones anteriores.

Al igual que antes, si se considera ahora que la cabeza experimenta diferentes movimientos en los diferentes ejes de giro en los instantes t=10s y t=25s como se muestra en la Figura 3.11 y 3.12 además de considerar los canales semicirculares ortogonales se obtienen los volúmenes desplazados mostrados en la Figura 3.25.

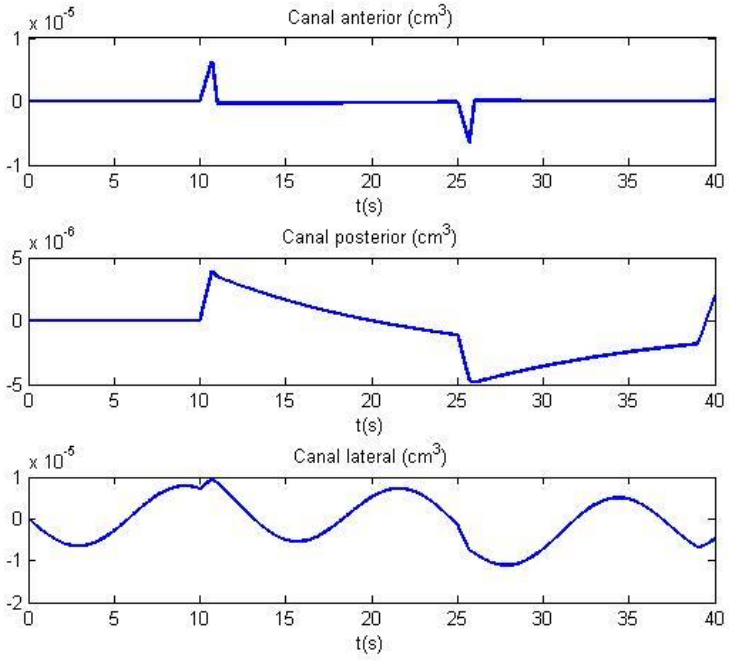


Figura 3.25 Volumen desplazado canales semicirculares ortogonales.

Se obtienen los mismos volúmenes desplazados en el canal anterior y posterior que en el caso de velocidad constante de la silla (Figura 3.25), sin embargo el canal lateral experimenta un cierto desplazamiento no senoidal en los intantes t=10s y t=25s debido al movimiento de la cabeza sobre el eje Z.

Si se consideran no ortogonales (aplicando el cambio al sistema R3) se obtiene la Figura 3.26.

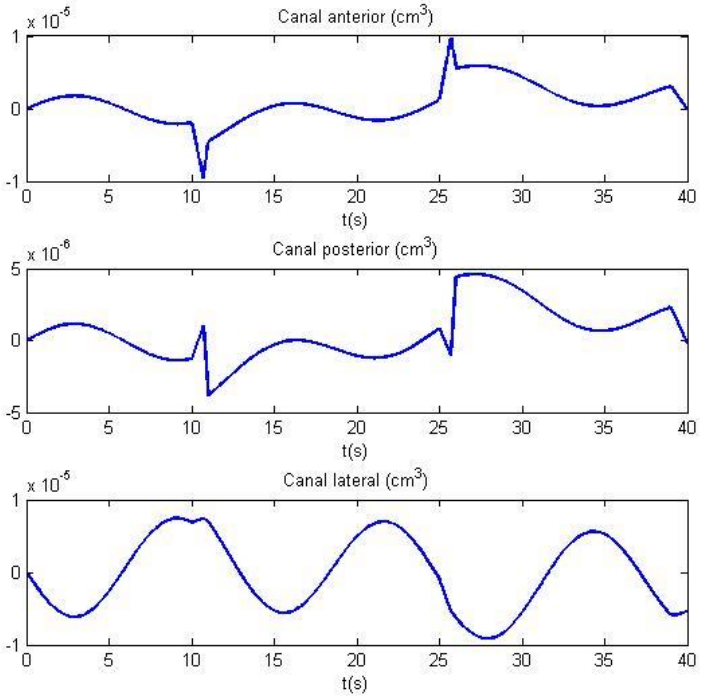


Figura 3.26 Volumen desplazado canales semicirculares no ortogonales.

Se observa que en los canales anterior y posterior el volumen desplazado se pierde lo forma senoidal debido a los giros de la cabeza (Figura 3.26), experimentando picos justo donde comienzan las aceleraciones (t=10s) y (t=25s), no obstante, los valores tienen los mismos órdenes de magnitud de casos anteriores. El canal lateral tiene la misma respuesta que en el caso de la Figura 3.25.

4 MODELADO Y SIMULACIÓN DEL ROBOT EN SIMULINK/SIMSCAPE

En este capítulo se va a construir un robot en SolidWorks para posteriormente implementarlo en Matlab/Simulink y simular el modelo cinemático del capítulo 2, utilizando como sensor de posición el modelo dinámico del canal semicircular.

Como se mencionó en la Introducción de este trabajo, para realizar el control de posición angular de un robot de tres grados de libertad, previo al desarrollo de su modelo en Simulink, se va a proceder a la creación de sus componentes (piezas) utilizando el software SolidWorks. El robot tiene el aspecto mostrado en la Figura 4.1.

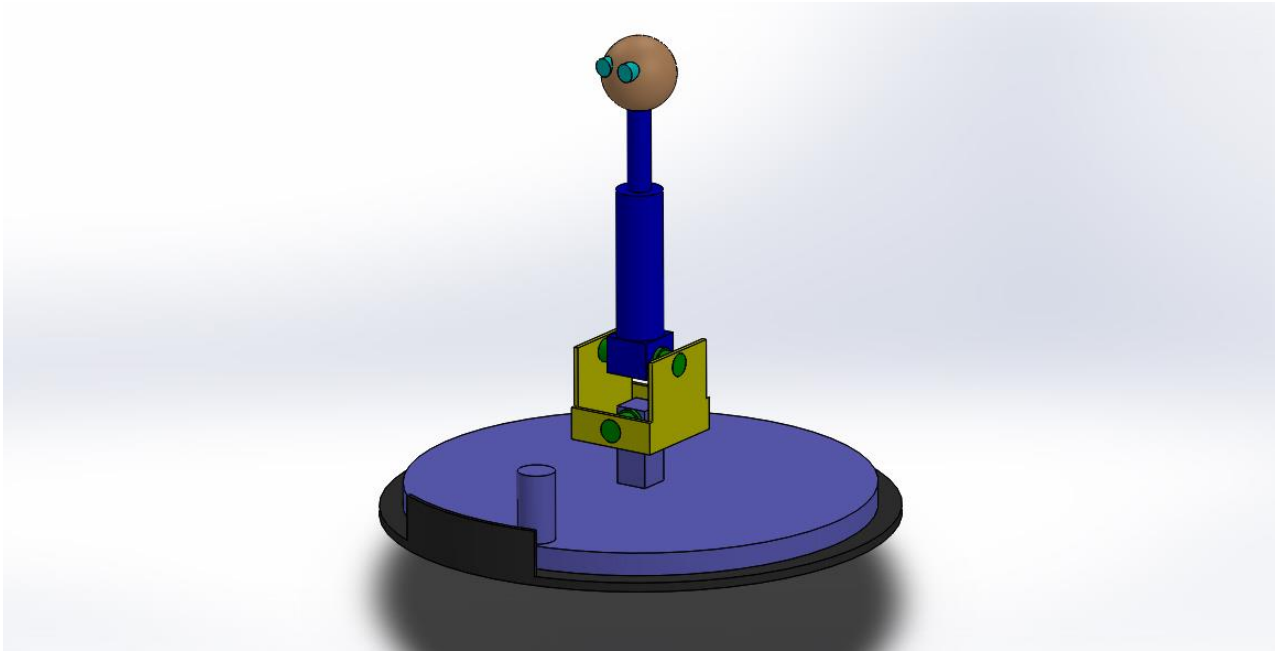


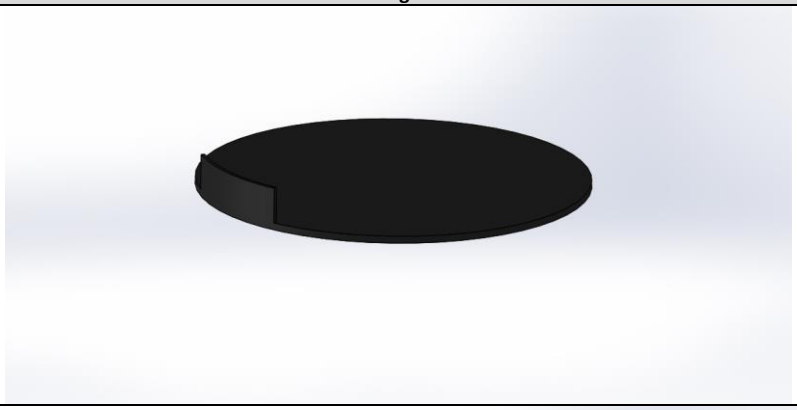
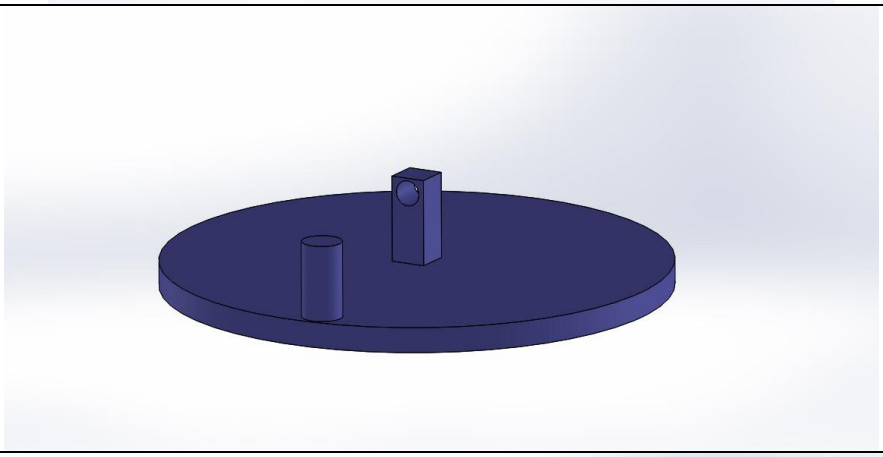
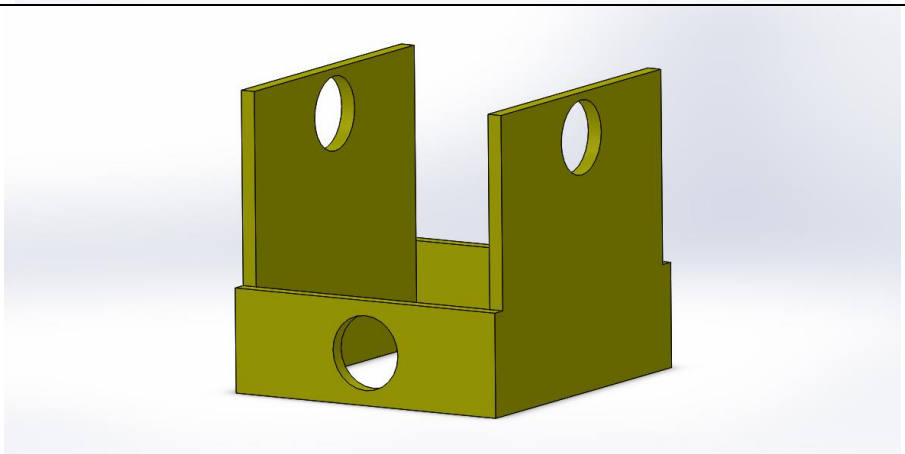
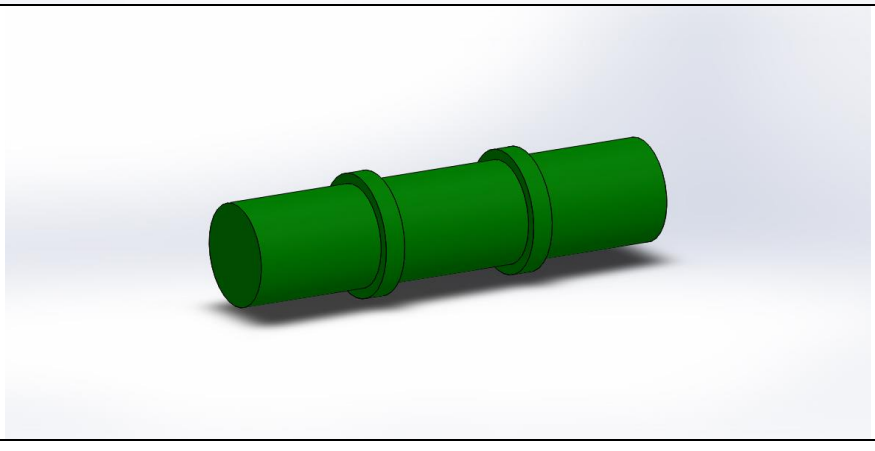
Figura 4.1 Robot de tres grados de libertad.

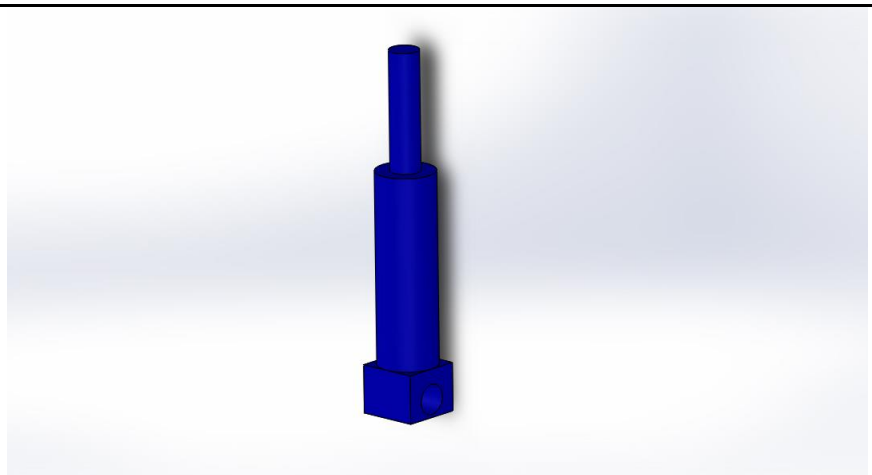
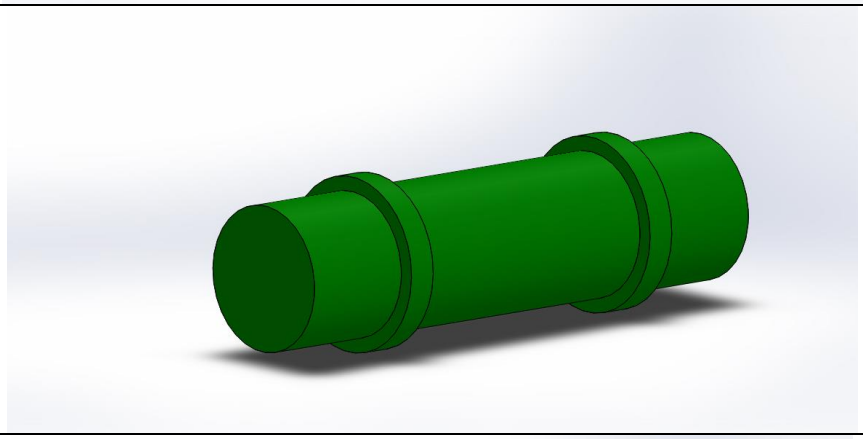
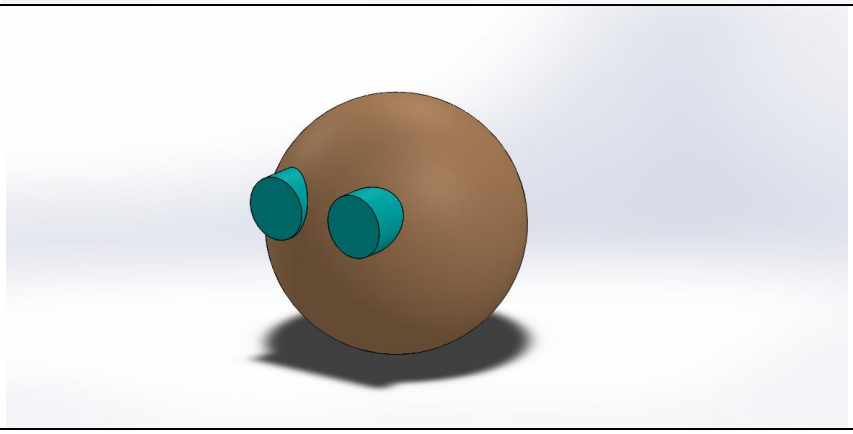
Como se puede ver en la Figura 4.1, se imita el modelo cinemático del capítulo 2 de este trabajo. El robot está sobre una plataforma giratoria y a su vez puede realizar tres giros emulando los giros de la cabeza (alabeo, cabeceo y guiñada). Para ello cuenta con dos articulaciones y a su vez la cabeza permite el giro relativo con el mástil donde va sujeta, es decir, cuenta con tres grados de libertad.

4.1 Creación de piezas en SolidWorks

El robot está formado por las piezas que se describen en la Tabla 4.1.

Tabla 4.1 Despiece del robot

Pieza	Imagen	Descripción
Base		Es la base de la plataforma giratoria. Es la parte del sistema que va unida al suelo.
Plataforma		Es el elemento que gira sobre la base. Sobre la plataforma se encuentra el robot unido mediante una articulación.
Soporte mástil		Es la pieza donde van unidas la plataforma y el mástil que sostiene la cabeza del robot. Tiene cuatro taladros donde van soportadas las dos primeras articulaciones.
Articulación X		Esta articulación une la plataforma giratoria con el soporte del mástil. Además permite el giro sobre el eje X (alabeo).

Mástil		El mástil es la pieza que soporta la cabeza del robot y a su vez, se une con el soporte del mástil.
Articulación Y		Esta articulación une el soporte del mástil con el propio mástil. Además permite el giro sobre el eje Y (cabeceo).
Cabeza		Es la cabeza del robot. Esta pieza gira sobre el mástil, es decir, gira sobre el eje Z (guiñada), lo que permite realizar el tercer grado de libertad.

Una vez que todas las piezas están creadas se procede al ensamblaje del conjunto de piezas. Dicho ensamblaje es el resultado que se puede apreciar en la Figura 4.1.

Posteriormente el ensamblaje se va a exportar a Simulink mediante la toolbox *Simscape Multibody* de Matlab.

4.2 Implementación en simulink/simscape

Como se ha comentado, se va a utilizar la toolbox de Matlab *Simscape Multibody*, la cual permite generar un modelo físico en Simulink a partir de un ensamblaje CAD, en este caso SolidWorks.

Este proceso comienza tal que, desde SolidWorks se guarda el archivo de ensamblaje con la extensión XML. Seguidamente en el *Command Window* de Matlab se utiliza el comando

mech_import('vestibular_3GDL.xml') en este caso y Matlab genera un modelo mecánico en Simulink a partir del archivo XML utilizando los elementos de *simmechanics*.

Antes de explicar el modelo obtenido, es necesario realizar el modelado de un motor de DC (motor de corriente continua), dado que se va a modelar el robot como si tuviera un motor de DC en cada articulación, y este será el actuador que hay que controlar para conseguir el control de posición angular mencionado en el punto 1 del presente trabajo.

4.2.1 Modelado del motor de DC

El funcionamiento básico de un motor de DC se muestra en la Figura 4.2.

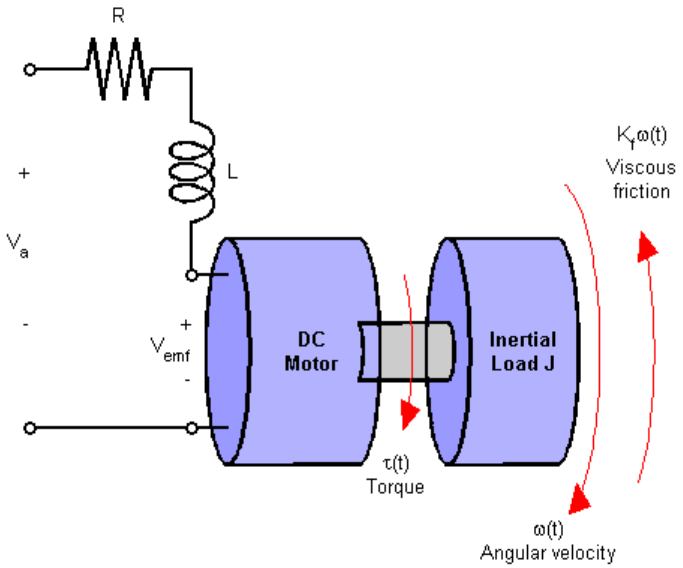


Figura 4.2 Motor de DC.

Una vez que se conoce el esquema se procede a modelar el motor, utilizando uno de los ejemplos que aporta Matlab. Para ello se ha utilizado un archivo m-file, cuyo código se puede consultar en el anexo correspondiente de este trabajo, en el que se definen todas las constantes del motor (R, L, constante de par, etc.) y se construye el modelo. El modelo en Simulink del motor tiene el aspecto mostrado en la Figura 4.3.

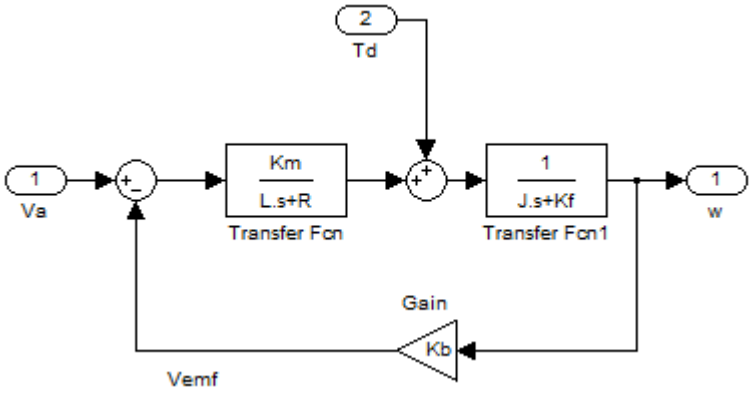


Figura 4.3 Modelo en Simulink del motor de DC.

Para que este modelo sea de utilidad para el robot tenemos que realizar una transformación de tal forma que la velocidad angular a la salida del motor sea la deseada. La entrada será una velocidad angular de consigna que mediante la correspondiente transformación pasará a voltios, como se muestra en la Figura 4.4.

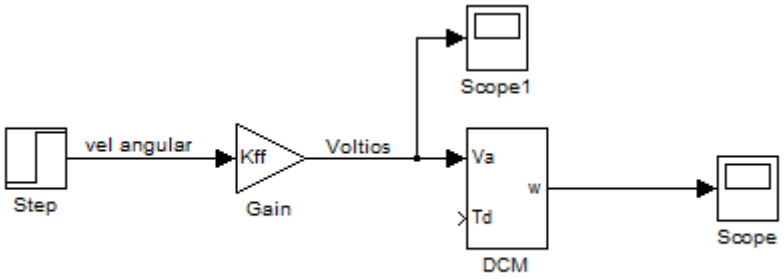


Figura 4.4 Modelo completo del motor más transformación a voltios.

Realizando una pequeña simulación para comprobar el funcionamiento, para una entrada escalón de amplitud 1 rad/s, se obtiene la velocidad angular a la salida mostrada en la Figura 4.5.

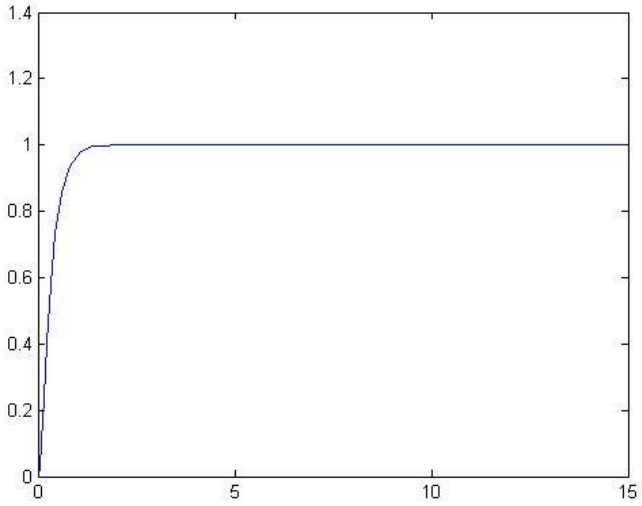


Figura 4.5 Velocidad angular (rad/s) dada por el motor de DC.

Para conseguir esta velocidad se necesita la tensión que se ve en la Figura 4.6.

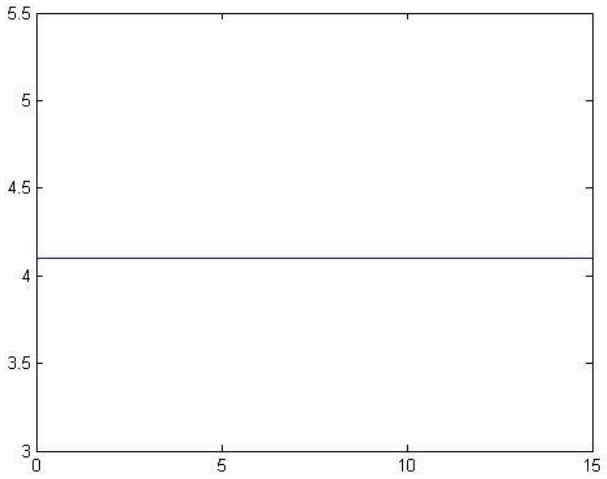


Figura 4.6 Tensión necesaria en voltios (V) para conseguir una velocidad angular de 1rad/s.

Se puede apreciar (Figura 4.6) que para conseguir una velocidad angular de 1rad/s a la salida del motor se necesita una tensión a la entrada de 4.1V.

Con el motor ya modelado y descrito se procede a explicar el modelo en Simulink del robot de tres grados de libertad.

4.2.2 Modelo completo

Para obtener el modelo del robot, como se ha mencionado antes, se utiliza el comando de Matlab *mech_import('vestibular_3GDL.xml')*. Este comando genera un modelo en Simulink utilizando los bloques de *simscape simmechanics*. Además se han agregado todos los elementos de control necesarios, así como el modelo del motor de DC y el modelo del canal semicircular como sensor de la posición angular. En la Figura 4.7 se puede observar el modelo completo.

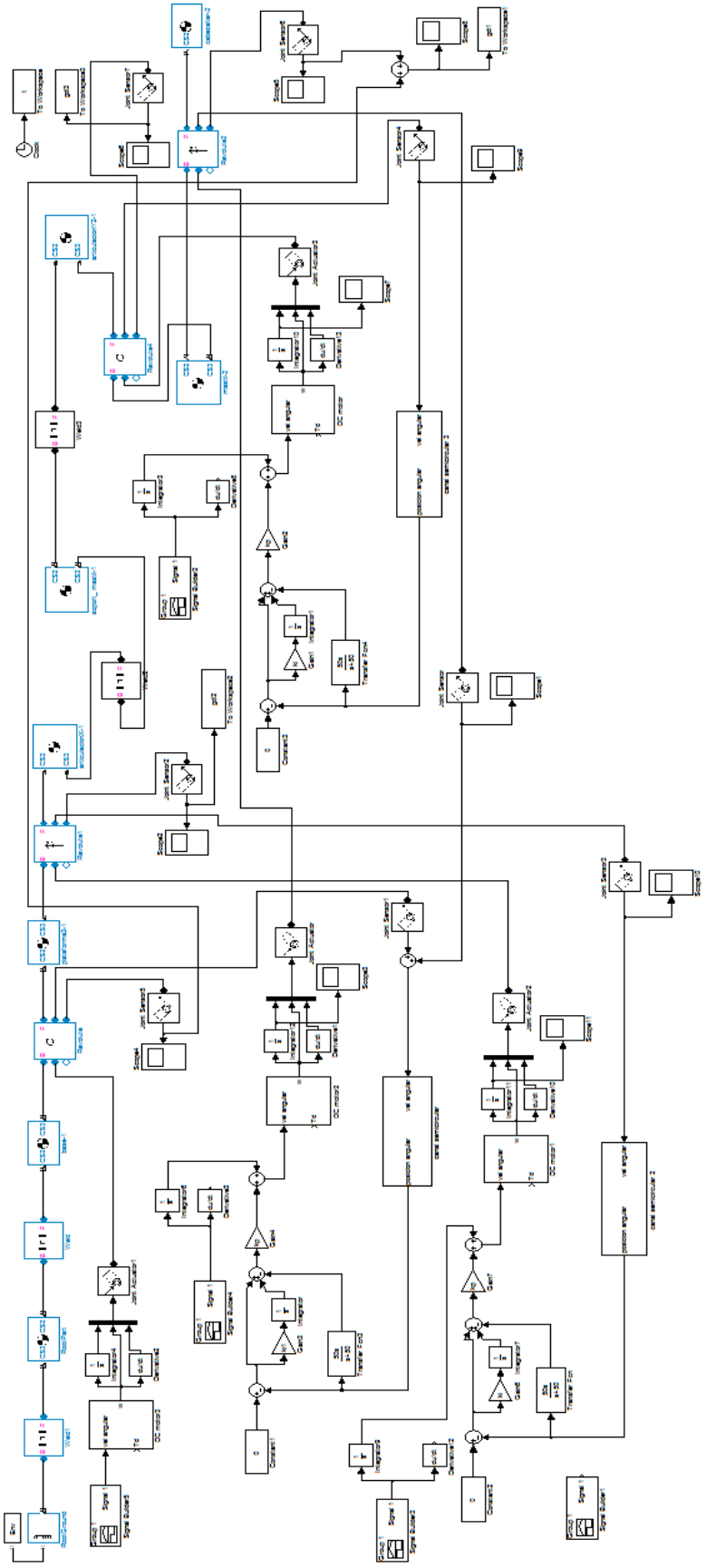


Figura 4.7 Modelo completo del robot en Simulink.

Se van a ir explicando las diferentes partes que componen el sistema, si bien, el control de posición angular de cada grado de libertad es igual.

En primer lugar se va a describir la parte mecánica, los bloques de *simmechanics* (bloques azules).

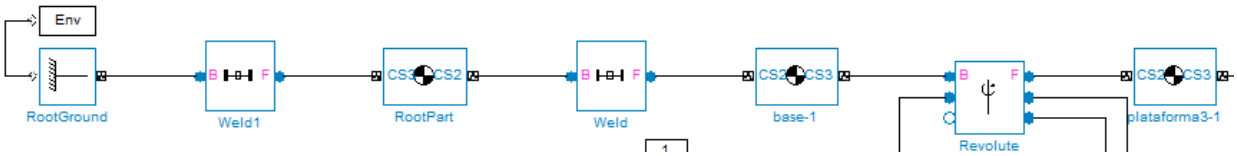


Figura 4.8 Bloques mecánicos.

En la Figura 4.8 se ve la primera parte del modelo mecánico. La base se considera fija y se une por una articulación de revolución a la plataforma, es decir, la plataforma girará relativa a la base.

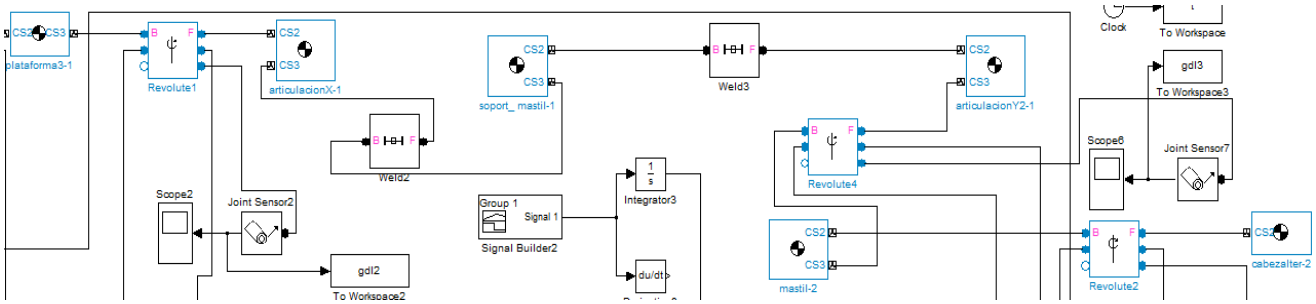


Figura 4.9 Bloques mecánicos.

Según se puede apreciar en la Figura 4.9, la plataforma está unida a la articulaciónX por un bloque revolución por lo que dicha articulación girará relativa a la plataforma. Seguidamente esa articulación está fija al soporte del mástil que a su vez está fijo con la articulaciónY. La articulaciónY está unida por un bloque de revolución al mástil, por lo que el mástil girará sobre esta articulación. Por último el mástil y la cabeza se unen por un bloque de revolución que permite el giro relativo de la cabeza sobre el mástil.

Seguidamente se puede ver en la Figura 4.10 como uno de los motores actúa sobre la plataforma para que gire:

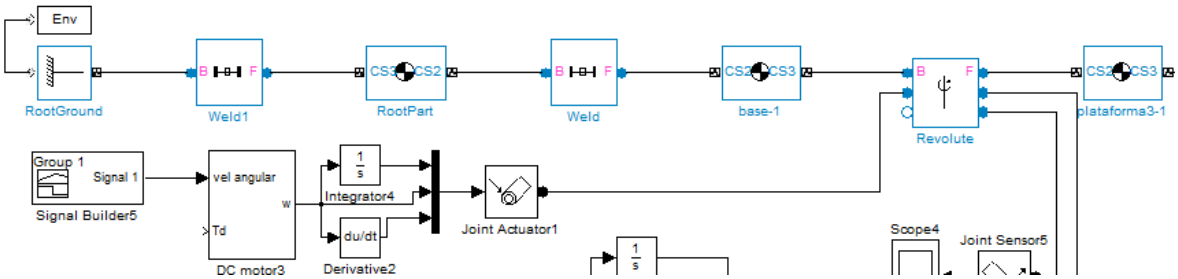


Figura 4.10 Actuación sobre la plataforma.

A partir de una velocidad angular definida, se actúa sobre el motor y se consigue la velocidad deseada a la salida del motor. Para actuar sobre el bloque revolución de *simmechanics* se utiliza un bloque previo llamado *joint actuator* que requiere como entrada el ángulo, la velocidad y la aceleración procedentes del motor. De esta forma se consigue el giro deseado de la plataforma.

Lo siguiente es describir el sistema de control que se utiliza para los diferentes grados de libertad. Hay que recordar que se va a hacer un control de la posición angular en lazo cerrado mediante un controlador PID, que la posición angular de consigna es 0 radianes en los tres grados de libertad y que la plataforma experimenta un giro al igual que el resto de articulaciones (se admite que el robot pueda tener giros sobre los tres ejes, al igual que la cabeza giraba sobre los tres ejes en el modelo cinemático descrito en el capítulo 2 de este trabajo). Los giros de las articulaciones, se añaden a modo de perturbación en el sistema en todos los lazos de control.

Para el grado de libertad del giro de la cabeza sobre el eje Z (guiñada) se tiene la Figura 4.11.

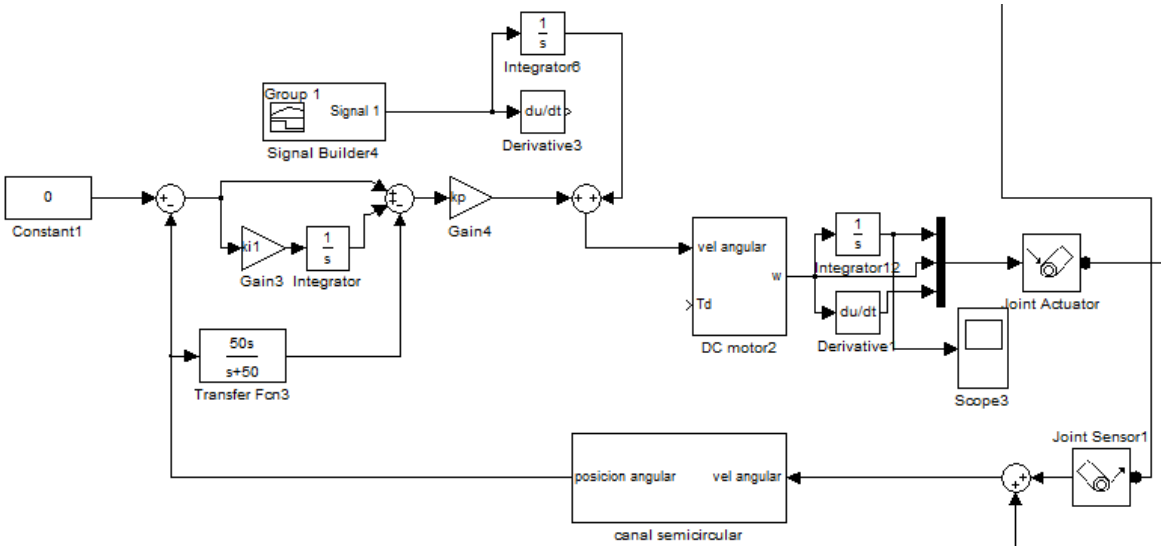


Figura 4.11 Control en lazo cerrado para el giro sobre Z.

Como se puede ver en la Figura 4.11 la velocidad angular que poseen tanto la plataforma como la propia cabeza (sobre el eje Z) en rad/s, que se obtienen a través de los bloques *joint sensor*, se suman debido a que este grado de libertad debe compensar el giro de la plataforma y el posible giro que tenga el robot sobre el eje Z a modo de perturbación. Una vez se tiene la velocidad angular que habría que compensar, se introduce en el subsistema *canal semicircular* el cual contiene el modelo del canal semicircular que se utiliza como sensor para medir la posición angular del respectivo grado de libertad. El subsistema mencionado tiene el aspecto de la Figura 4.12.

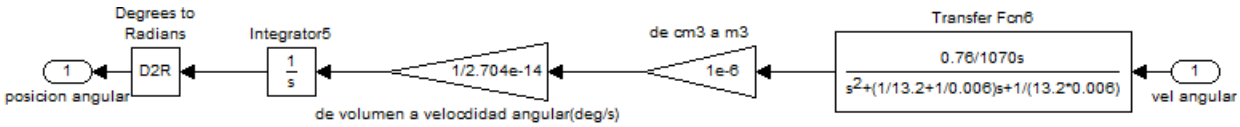


Figura 4.12 Subsistema *canal semicircular*.

Utilizando el modelo del canal semicircular de la ecuación (3.7), introduciendo la velocidad angular de la cabeza del robot más la de la plataforma podemos obtener el volumen de endolinfa desplazado por la cúpula en cm3. Pasando este volumen por la ganancia que se ve en la Figura 4.12, se obtiene la velocidad angular en grados/s, que integrando seguidamente da lugar a la posición angular en grados. Finalmente pasando a radianes se obtiene la posición angular a controlar.

Cerrando el lazo de control esta posición angular conseguida se compara con la de consigna, para conseguir la señal de error, entrada del controlador como se muestra en la Figura 4.11.

Se ha optado por implementar un controlador PID, cuya parte derivativa cuenta con un filtro paso bajo. Se van a proponer valores de las partes proporcional, integral y derivativa, de forma empírica. Además se compararán los resultados con diferentes valores propuestos. A la señal de control (posición angular en radianes), antes de entrar al modelo del motor de DC (modelo obtenido en el punto 4.2.1), se le suma la perturbación (posición angular en radianes) comentada anteriormente que emula el movimiento de la cabeza y luego esta señal entra al motor. A pesar de que el modelo del motor requería de una velocidad angular a la entrada, también funciona introduciéndole como entrada una posición angular. Esto es así porque esta posición angular se integra (la velocidad angular se obtiene de los bloques *joint sensor* como se ha visto anteriormente) para que entre como velocidad angular en rad/s al modelo del canal semicircular por lo que el efecto es el mismo.

El control en lazo cerrado es el mismo para los grados de libertad del giro de la cabeza sobre los ejes X e Y (alabeo y cabeceo). El lazo de control sería el mostrado en la Figura 4.13:

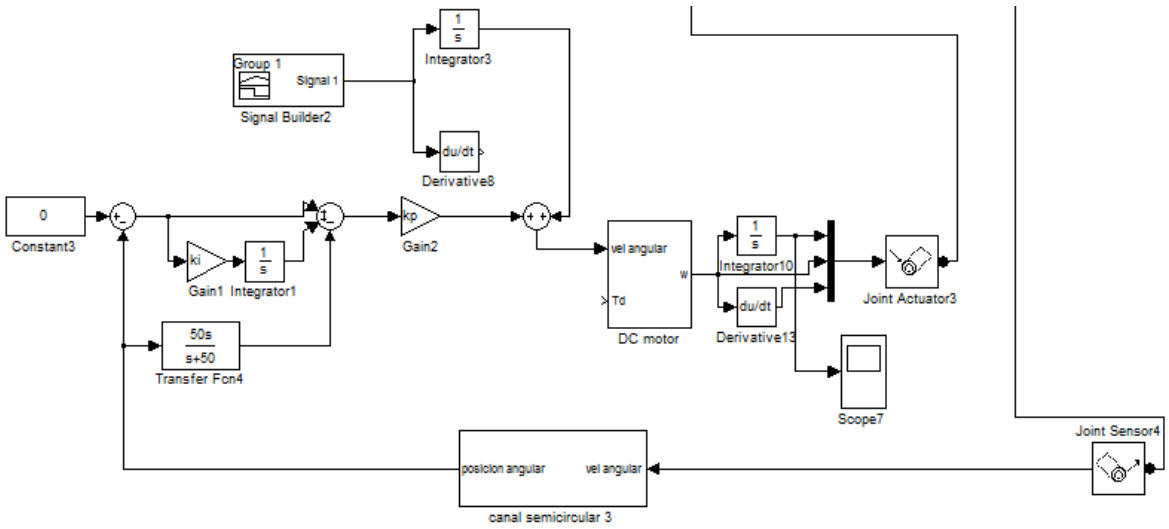


Figura 4.13 Control en lazo cerrado para los sobre X e Y.

Como se puede apreciar en la Figura 4.13, el lazo de control es igual que en el caso del grado de libertad en el giro sobre Z, salvo porque en los giros sobre X e Y solo se debe compensar la perturbación de la señal de control (movimiento del robot en esos ejes), es decir, no hay ninguna otra contribución a estos giros.

4.3 Simulaciones

Se va a llevar a cabo una simulación del modelo en la que se puede visualizar una cinemática del robot y también se van a obtener las señales de error en términos de posición angular en grados.

La velocidad angular a la que se va a someter la plataforma es la que se puede ver en la Figura 4.14.

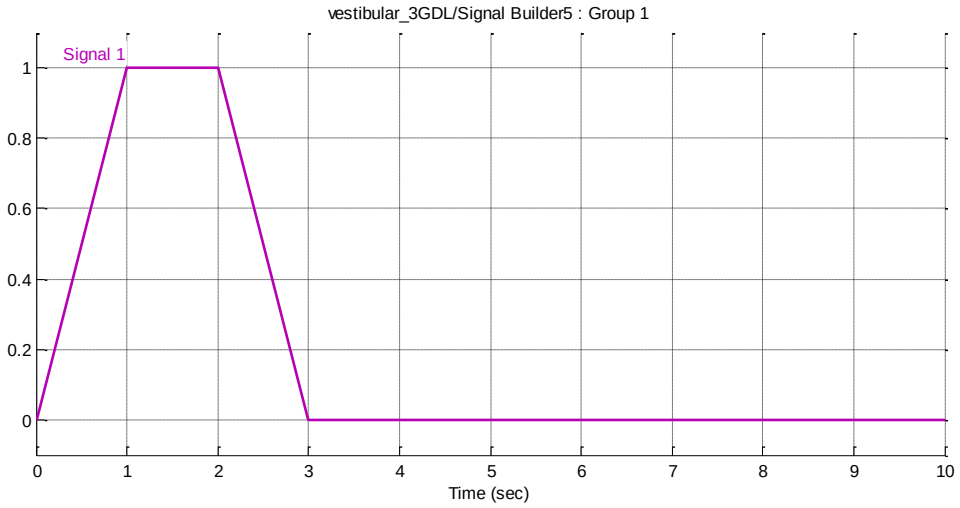


Figura 4.14 Velocidad angular de la plataforma (rad/s).

Las velocidades angulares de los giros del robot sobre los ejes X, Y, Z (perturbaciones) son iguales y tienen la función mostrada en la Figura 4.15.

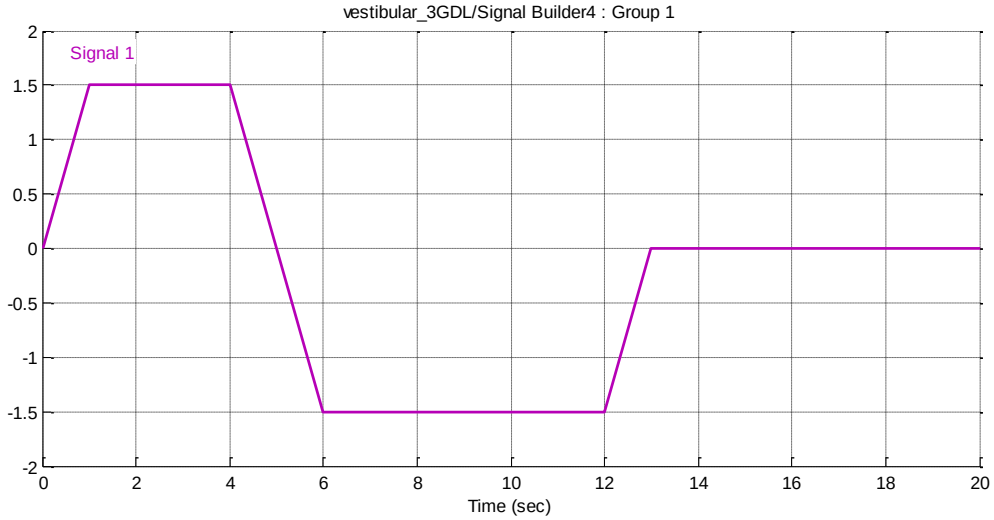


Figura 4.15 Velocidad angular del robot en los ejes X, Y, Z (rad/s).

Esta velocidad angular se integrará para pasarla a posición angular antes de sumarla como perturbación a la señal de control.

Para el controlador PID (Figura 4.11 y 4.13) se van a utilizar los valores de la constantes Kp=2, Ki=5, Ki1=5, Kd=50 y N=50. Siendo los Ki1 el valor de la parte integral del controlador del grado de libertad en el eje Z. Ki es el valor de la parte integral para los grados de libertad X e Y. Kp es común para los tres grados de libertad. La parte derivativa, Kd, es común para los tres grados de libertad y cuenta con un filtro de valor N=50.

Se obtiene la señal de error de la Figura 4.16 en grados, en cada uno de los tres grados de libertad.

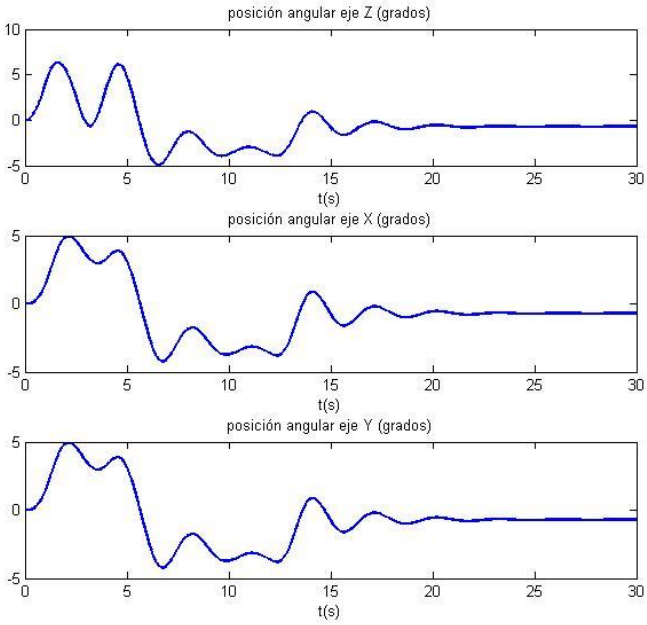


Figura 4.16 Señal de error en grados en cada eje.

Se puede ver (Figura 4.16) como el controlador trata de llevar el error a 0 grados y cuando cesan los giros consigue que el error tenga un valor estable, aunque existe un pequeño error estacionario por el que no se alcanza los 0 grados, debido a las perturbaciones introducidas. La señal en el eje Z se obtiene sumando el ángulo recorrido por la plataforma más el ángulo recorrido por la cabeza (que tendrá signo contrario), dado que este grado de libertad debe compensar el giro de la plataforma, el resultado de esta suma debe ser cero o muy próximo a cero.

Si cambiamos los valores de las constantes del controlador por Kp=2, Ki=8 y Ki1=8 se obtiene la Figura 4.17.

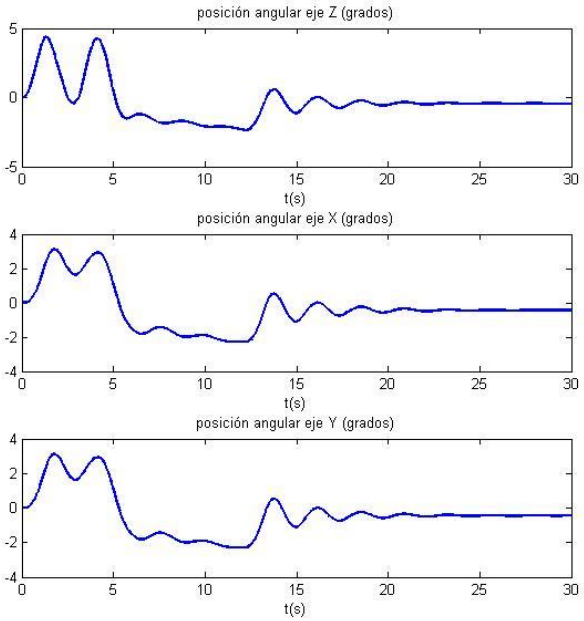


Figura 4.17 Señal de error en grados en cada eje.

Con estos valores del controlador se obtiene un menor error estacionario (Figura 4.17), además de un menor sobreimpulso de la señal de error.

Con la simulación del modelo además de obtener las señales de error se obtiene una cinemática del robot como se había comentado antes. El aspecto de la cinemática es el de la Figura 4.18.

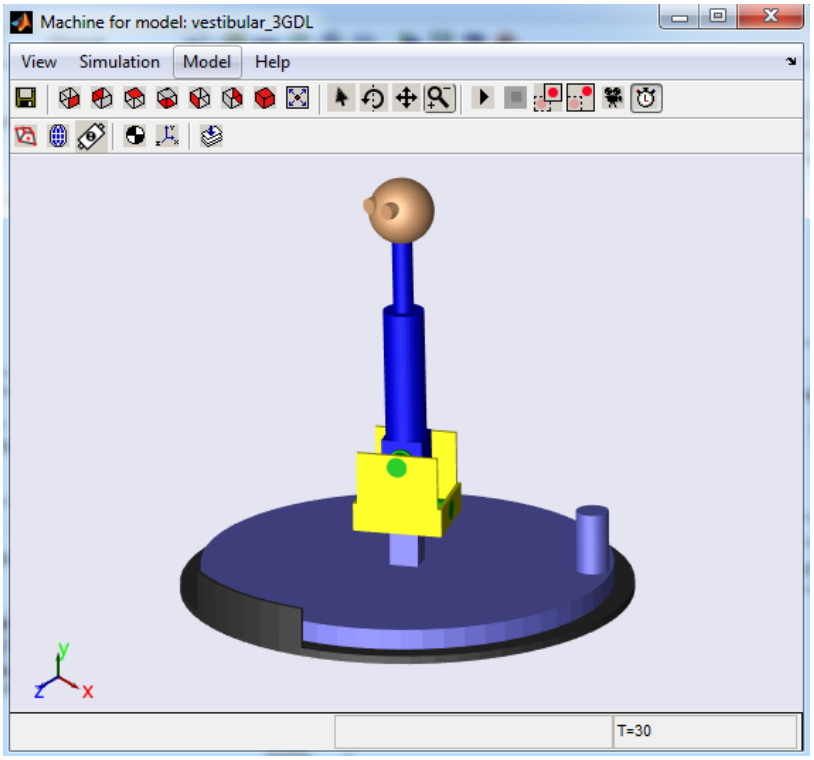


Figura 4.18 Cinemática del robot en Simulink.

Tras la simulación se puede concluir, que el modelo dinámico del canal semicircular es perfectamente válido como sensor de posición angular, dado que se ha conseguido realizar un seguimiento y control de la posición angular.

5 INTEGRACIÓN DE MODELOS: INTERFAZ GRÁFICA DE USUARIO

Se ha decidido crear una interfaz gráfica con Matlab para integrar tanto el modelo del sistema vestibular, como el modelo del robot en un único lugar. Además el usuario podrá interactuar con la interfaz a través de diferentes menús desplegables y demás elementos de interacción. La interfaz gráfica tiene el aspecto mostrado en la Figura 5.1.

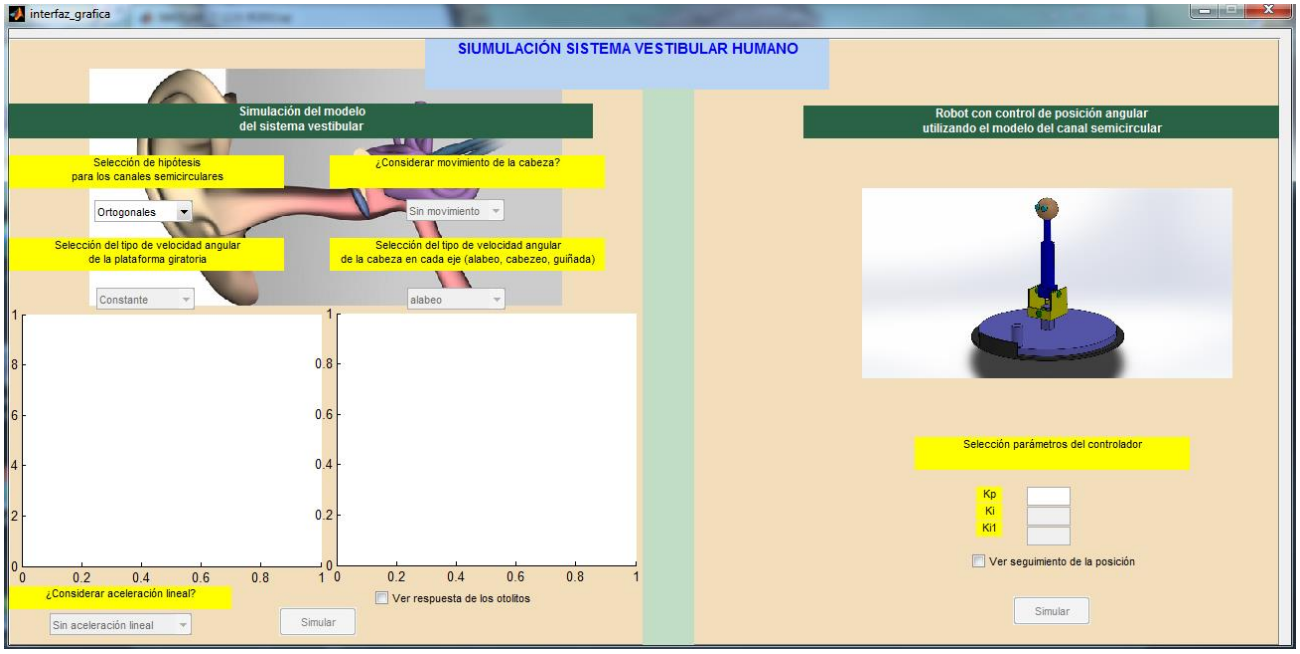


Figura 5.1 Interfaz gráfica de usuario.

Conforme a la Figura 5.1, en la parte del modelo del sistema vestibular, el usuario deberá seleccionar con los menús desplegables si desea hacer la simulación con la consideración de canales semicirculares ortogonales o no ortogonales; el tipo de velocidad angular de la silla; si se desea o no considerar movimiento de la cabeza; el tipo de velocidad de la cabeza; si desea añadir vector de velocidad lineal o no; y si se desea visualizar la respuesta de los otolitos. Posteriormente deberá pulsar el botón *Simular* y la simulación se ejecutará con todos los parámetros seleccionados.

Al seleccionar los tipos de velocidad angular comentados, en los plots que se pueden ver en la Figura 5.1, aparecerán las funciones de velocidad angular seleccionadas.

Por otro lado en la parte de simulación del robot, el usuario podrá seleccionar las constantes del controlador del robot y si desea o no visualizar las señales de error. Posteriormente pulsará el botón *Simular* y la simulación se ejecutará con los parámetros seleccionados. Además aparecerá la cinemática del robot al pulsar dicho botón.

De esta forma todo lo descrito en el presente trabajo queda integrado en un único lugar desde el que se puede gobernar todo el sistema, seleccionando los diferentes parámetros y aportando así una mayor comodidad y ahorro de tiempo al usuario.

6 CONCLUSIONES

A lo largo de este trabajo y tras haber realizado un modelado cinemático, haber obtenido el modelo dinámico de los canales semicirculares, los otolitos y habiendo simulado el modelo completo del sistema vestibular, se puede concluir que el sistema vestibular humano funciona como un sensor de los movimientos angulares (canales semicirculares) y los movimientos de traslación (otolitos), de tal forma que con toda la información recabada por el sistema vestibular que llega al cerebro, el cuerpo humano puede utilizar esta información para mantener el equilibrio, percibir el entorno y controlar su posición. Lo cual demuestra la gran importancia del sistema vestibular para la vida humana.

Por otro lado, en lo que respecta al control de posición angular del robot realizado en este trabajo, se puede concluir que, efectivamente, el modelo dinámico del canal semicircular es válido para utilizarlo como sensor de posición angular y se ha conseguido realizar el control de posición con éxito.

En lo que respecta a posibles líneas de trabajo futuro del presente, la más inmediata sería la construcción real del robot y la utilización del modelo que se ha desarrollado para aportar las señales de control a los motores eléctricos reales. Además se puede ajustar el modelo del motor de DC al motor real que se tenga disponible. Habría que decidir qué materiales utilizar para fabricar las piezas del robot, puesto que también se deberían tener en cuenta las masas de las piezas de cara al cálculo de las inercias y ver qué motores serían válidos.

Si ya se dispone de una plataforma giratoria, solo haría falta adquirir tres motores eléctricos, además de las piezas del robot. Finalmente se deberían conectar los motores y mediante el ordenador, utilizando el modelo en Simulink del robot, realizar el control.

7 BIBLIOGRAFÍA

[1] P.Selva, *Modeling of the vestibular system and nonlinear models for human spatial orientation perception*, 29 Septiembre 2009.

[2] P.Selva, J.Morlier, Y.Gourinat, *Development of a Dynamic Virtual RealityModel of the Inner Ear Sensory Systemas a Learning and Demonstrating Tool*, 9 Octubre 2008.

[3] J.García-Valdecasasl, A.Aviñoa, C.Arjona, *FISIOLOGÍA DEL SISTEMA VESTIBULAR*.

[4] http://www.clinicajuancarrero.es/disciplinas/otoneurologia/10-isciplinas/otoneurologia/35-anatomia-del-sistema-vestibular

[5] https://beautifulsci.wordpress.com/tag/oido-equilibrio-sentidos-animal/

[6] https://es.mathworks.com/help/control/examples/dc-motor-control.html

[7] R.Rabbitt, E.Damiano, J.Grant, *The Vestibular System*, Springer, 2004.

[8] R.Telban, F.Cardullo, *Motion Cueing Algorithm Development: Human-Centered Linear and Nonlinear Approaches*, NASA/CR-2005-213747, Mayo 2005.

8 ANEXO: CÓDIGOS DE MATLAB

Se exponen a continuación los códigos de los diferentes archivos m-file y funciones de Matlab programadas en este trabajo.

8.1 constantesvestibular.m

```
% Constantes y Variables sistema vestibular

m = 1070;   % Endolymph mass within the horizontal canal
assignin('base','m',m);

d = 0.76;   % Inertial forcing coefficient
assignin('base','d',d);

tau1 = 13.2;    % Slow time constant of the horizontal semicircular canal
assignin('base','tau1',tau1);

tau2 = 0.006;    % Fast time constant of the horizontal semicircular canal
assignin('base','tau2',tau2);

% Simulaciones y plots de resultados
sim('vestibular_system_final')

% Canales semicirculares
figure
subplot(3,1,1)
plot(t,desp_cupula_anterior,'lineWidth',2)
title('Canal anterior (cm^3)')
xlabel('t(s)')
subplot(3,1,2)
plot(t,desp_cupula_posterior,'lineWidth',2)
title('Canal posterior (cm^3)')
xlabel('t(s)')
subplot(3,1,3)
plot(t,desp_cupula_lateral,'lineWidth',2)
title('Canal lateral (cm^3)')
xlabel('t(s)')

global plot1;
assignin('base','plot1',plot1);
% Otolitos
if plot1 == 1
    figure(2)
    subplot(3,1,1)
    plot(t,x_otolito,'lineWidth',2)
    title('Afferent Firing Rate (AFR) (eje X)')
    xlabel('t(s)')
    subplot(3,1,2)
    plot(t,y_otolito,'lineWidth',2)
    title('Afferent Firing Rate (AFR) (eje Y)')
    xlabel('t(s)')
    subplot(3,1,3)
    plot(t,z_otolito,'lineWidth',2)
    title('Afferent Firing Rate (AFR) (eje Z)')
    xlabel('t(s)')
else
end
```

8.2 MATLAB Function1 (figura 2.5)

```
function [w1R1,w1R0,VB1R1,VB1R0,accw1R1,accw1R0,delta,Mdelta]=
fcn(omega1,AB,wpre,tpre,t)
%#codegen
%velocidad angular de la silla (rad/s) en R2
%w1R2 = MR2*omega1;

%velocidad angular de la silla (rad/s) en R1
w1R1 = omega1;

%ángulo delta de la silla respecto al sistema R0
%delta = int(omega1(3),0,t); ángulo recorrido por la silla si la vel
%angular es una función
delta = omega1(3)*(t-tpre); %ángulo recorrido por la silla si la vel
%angular es un valor numérico

Mdelta=[cos(delta) sin(delta) 0; -sin(delta) cos(delta) 0; 0 0 1];

%velocidad angular de la silla (rad/s) en R0
w1R0 = Mdelta*omega1;

%aceleración angular de la silla en R1
%w1acc = [0 0 0]; para velocidad constante de la silla
%accw1R1 = diff(omega1(3),t); aceleración angular si tuvieramos una función
%de la velocidad
% Aceleración angular de la silla en R1
if t == 0
  accw1R1 = [0; 0; 0];
else
    accw1R1 = (omega1-wpre)/(t-tpre); % aceleración angular si tenemos un valor
numérico de la velocidad angular
end
% Aceleración angular de la silla en R0
accw1R0 = Mdelta*accw1R1;

%velocidad lineal del punto B en R1 debido a la velocidad de la silla
VB1R1 = cross(w1R1,AB);

%velocidad lineal de la silla en R0 debido a la velocidad de la silla
VB1R0 = cross(w1R0,AB);
```

8.3 MATLAB Function (figura 2.6)

```
function
[w2R2,w2R1,w2R0,VB2R2,VB2R1,VB2R0,accw2R2,accw2R1,accw2R0,alpha,beta,gamma,Malph
a,Mbeta,Mgamma,MR2] = fcn(omega2,AB,wpre,tpre,Mdelta,t)
%#codegen

%velocidad angular de la cabeza (los 3 ejes) en R2
w2R2 = omega2;

%ángulo roll
alpha = omega2(1)*(t-tpre);

%ángulo pitch
beta = omega2(2)*(t-tpre);
```

```
%ángulo yaw
gamma = omega2(3)*(t-tpre);

%matrices de cambio de sistema de referencia
Malpha = [1 0 0; 0 cos(alpha) sin(alpha); 0 -sin(alpha) cos(alpha)];
Mbeta = [cos(beta) 0 -sin(beta); 0 1 0; sin(beta) 0 cos(beta)];
Mgamma = [cos(gamma) sin(gamma) 0; -sin(gamma) cos(gamma) 0; 0 0 1];
MR2 = Malpha*Mbeta*Mgamma;

%velocidad angular de la cabeza en R1
w2R1 = Malpha*Mbeta*Mgamma*omega2;

%velocidad angular de la cabeza en R0
w2R0 = Malpha*Mbeta*Mgamma*Mdelta*omega2;

% Aceleración angular de la cabeza en R2
if t == 0
    accw2R2 = [0; 0; 0];
else
    accw2R2 = (omega2-wpre)/(t-tpre);
end

% Aceleración angular de la cabeza en R1
accw2R1 = MR2*accw2R2;

% Aceleración angular de la cabeza en R0
accw2R0 = MR2*Mdelta*accw2R2;

%velocidad lineal del punto B en R2 debido a la velocidad de la cabeza
VB2R2 = cross(w2R2,AB);

%velocidad lineal del punto B en R1 debido a la velocidad de la cabeza
VB2R1 = cross(w2R1,AB);

%velocidad lineal del punto B en R0 debido a la velocidad de la cabeza
VB2R0 = cross(w2R0,AB);
```

8.4 MATLAB Function2 (figura 2.7)

```
function
[w20R0,w20R1,w20R2,VB20R2,VB20R1,VB20R0,AB2R0,AB1R0,Acent,Acori,AB20R0]=
fcn(w1R0,w2R0,w1R1,w2R1,w2R2,AB,VB1R1,VB1R0,VB2R2,VB2R1,VB2R0,accw2R0,accw1R0,MR
2)
%#codegen

% Velocidad angular de la contribución silla+cabeza en el sistema de referencia
earth (R0)
w20R0 = w2R0+w1R0;

% Velocidad angular de la contribución silla+cabeza en R1
w20R1 = w2R1+w1R1;

% Velocidad angular de la contribución silla+cabeza en R2
w1R2 = MR2*w1R1;
w20R2 = w2R2+w1R2;

% velocidad lineal del punto B en R2 debido a la velocidad de la cabeza +
```

```
% silla
VB1R2 = cross(w1R2,AB);
VB20R2 = VB2R2+VB1R2;

% velocidad lineal del punto B en R1 debido a la velocidad de la cabeza +
% silla
VB20R1 = VB2R1+VB1R1;

% velocidad lineal del punto B en R0 debido a la velocidad de la cabeza +
% silla
VB20R0 = VB2R0+VB1R0;

% Aceleraciones lienales (4 términos)
% primer término
AB2R0 = cross(accw2R0,AB);

% segundo término
AB1R0 = cross(accw1R0,AB);

% tercer término (aceleración centrípeta)
Acent = cross(w1R0,cross(w1R0,AB));

% cuarto término (aceleración de coriolis)
Acori = 2*cross(w1R0,VB2R0);

% Aceleración absoluta
AB20R0 = AB2R0+AB1R0+Acent+Acori;
```

8.5 MATLAB Function3 (figura 2.8)

```
function [accw20R0,accw20R3] = fcn(accw1R0,accw2R0)
%#codegen

% Aceleración angular de la contribución silla+cabeza en el sistema earth
% (R0)
accw20R0 = accw2R0+accw1R0;

% ángulos de Euler canales semicirculares (rad)

chia=2.212;
chip=2.336;
chil=0;
thetaa=0.177;
thetap=0;
thetal=-0.331;
fia=0;
fip=-0.274;
fil=0.038;

%matriz de trasformación a R3

MR3=[cos(thetaa)*cos(chia) cos(thetaa)*sin(chia) -sin(thetaa); -
cos(fip)*sin(chip) cos(fip)*cos(chip) sin(fip); cos(fil)*sin(thetal) -sin(fil)
cos(fil)*cos(thetal)];

% Aceleración angular de la contribución silla+cabeza en el sistema de los
% canales semicirculares (R3)
% accw20R3 = (MR3*accw2R0)+(MR3*accw1R0);
```

```
accw20R3 = MR3*accw20R0;
```

8.6 constantes_robot.m

```
% Robot con control de posición mediante canal semicircular

% Creación del modelo del motor de corriente continua
% DC motor model
R = 2.0;                % Ohms (2 ohmios la original)
assignin('base','R',R);
L = 0.5;                % Henrys
assignin('base','L',L);
Km = 0.1;               % torque constant
assignin('base','Km',Km);
Kb = 0.1;               % back emf constant
assignin('base','Kb',Kb);
Kf = 0.2;               % Nms
assignin('base','Kf',Kf);
J = 0.02;               % kg.m^2/s^2
assignin('base','J',J);
% Td = -0.1 * (t>5 & t<10);       % load disturbance

% Construcción del modelo
h1 = tf(Km,[L R]);            % armature
assignin('base','h1',h1);
h2 = tf(1,[J Kf]);            % eqn of motion
assignin('base','h2',h2);
dcm = ss(h2) * [h1 , 1];      % w = h2 * (h1*Va + Td)
assignin('base','dcm',dcm);
dcm = feedback(dcm,Kb,1,1);   % close back emf loop

Kff = 1/dcgain(dcm(1));        % ganancia del sistema
assignin('base','Kff',Kff);

%Parámetros del controlador
% ki = 8;
% assignin('base','ki',ki);
% kp = 2;
% assignin('base','kp',kp);
% ki1 = 5;
% assignin('base','ki1',ki1);

% Simulación del robot
open_system('vestibular_3GDL')
sim('vestibular_3GDL')

global plot2;
assignin('base','plot2',plot2);
% Plots
if plot2 == 1
    figure
    subplot(3,1,1)
    plot(t,gdl1,'lineWidth',2)
    title('posición angular eje Z (grados)')
    xlabel('t(s)')
    subplot(3,1,2)
    plot(t,gdl2,'lineWidth',2)
    title('posición angular eje X (grados)')
    xlabel('t(s)')
```

```
    subplot(3,1,3)
    plot(t,gdl3,'lineWidth',2)
    title('posición angular eje Y (grados)')
    xlabel('t(s)')
else
end
```

8.7 interfaz_grafica.m

```
function varargout = interfaz_grafica(varargin)
% INTERFAZ_GRAFICA MATLAB code for interfaz_grafica.fig
%      INTERFAZ_GRAFICA, by itself, creates a new INTERFAZ_GRAFICA or raises the
existing
%      singleton*.
%
%      H = INTERFAZ_GRAFICA returns the handle to a new INTERFAZ_GRAFICA or the
handle to
%      the existing singleton*.
%
%      INTERFAZ_GRAFICA('CALLBACK',hObject,eventData,handles,...) calls the
local
%      function named CALLBACK in INTERFAZ_GRAFICA.M with the given input
arguments.
%
%      INTERFAZ_GRAFICA('Property','Value',...) creates a new INTERFAZ_GRAFICA
or raises the
%      existing singleton*.  Starting from the left, property value pairs are
%      applied to the GUI before interfaz_grafica_OpeningFcn gets called.  An
%      unrecognized property name or invalid value makes property application
%      stop.  All inputs are passed to interfaz_grafica_OpeningFcn via varargin.
%
%      *See GUI Options on GUIDE's Tools menu.  Choose "GUI allows only one
%      instance to run (singleton)".
%
% See also: GUIDE, GUIDATA, GUIHANDLES

% Edit the above text to modify the response to help interfaz_grafica

% Last Modified by GUIDE v2.5 10-Jul-2017 13:28:10

% Begin initialization code - DO NOT EDIT
gui_Singleton = 1;
gui_State = struct('gui_Name',       mfilename, ...
                   'gui_Singleton',  gui_Singleton, ...
                   'gui_OpeningFcn', @interfaz_grafica_OpeningFcn, ...
                   'gui_OutputFcn',  @interfaz_grafica_OutputFcn, ...
                   'gui_LayoutFcn',  [] , ...
                   'gui_Callback',   []);
if nargin && ischar(varargin{1})
    gui_State.gui_Callback = str2func(varargin{1});
end

if nargout
    [varargout{1:nargout}] = gui_mainfcn(gui_State, varargin{:});
else
    gui_mainfcn(gui_State, varargin{:});
end
% End initialization code - DO NOT EDIT
```

```
% --- Executes just before interfaz_grafica is made visible.
function interfaz_grafica_OpeningFcn(hObject, eventdata, handles, varargin)
% This function has no output args, see OutputFcn.
% hObject    handle to figure
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    structure with handles and user data (see GUIDATA)
% varargin   command line arguments to interfaz_grafica (see VARARGIN)

% Choose default command line output for interfaz_grafica
handles.output = hObject;

% Update handles structure
guidata(hObject, handles);
set(handles.pushbutton1,'enable','off')
set(handles.popupmenu2,'enable','off')
set(handles.popupmenu3,'enable','off')
set(handles.popupmenu4,'enable','off')
set(handles.popupmenu5,'enable','off')
set(handles.pushbutton2,'enable','off')
set(handles.edit2,'enable','off')
set(handles.edit3,'enable','off')
axes(handles.axes1)
imshow('vestibular.jpeg')
axes(handles.axes2)
imshow('robot_foto.jpg')
% axes(handles.axes7)
% imshow('vestibular.jpg')

% UIWAIT makes interfaz_grafica wait for user response (see UIRESUME)
% uiwait(handles.figure1);


% --- Outputs from this function are returned to the command line.
function varargout = interfaz_grafica_OutputFcn(hObject, eventdata, handles)
% varargout  cell array for returning output args (see VARARGOUT);
% hObject    handle to figure
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    structure with handles and user data (see GUIDATA)

% Get default command line output from handles structure
varargout{1} = handles.output;


% --- Executes on button press in pushbutton1.
function pushbutton1_Callback(hObject, eventdata, handles)
% hObject    handle to pushbutton1 (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    structure with handles and user data (see GUIDATA)
constantesvestibular
% global canal;
% assignin('base','canal',canal);
% canal = get(hObject,'Value')-1

% --- Executes on selection change in popupmenu1.
function popupmenu1_Callback(hObject, eventdata, handles)
% hObject    handle to popupmenu1 (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    structure with handles and user data (see GUIDATA)
```

```
% Hints: contents = cellstr(get(hObject,'String')) returns popupmenu1 contents
as cell array
%        contents{get(hObject,'Value')} returns selected item from popupmenu1
global canal;
canal = get(hObject,'Value')-1
assignin('base','canal',canal);
set(handles.popupmenu2,'enable','on')

% --- Executes during object creation, after setting all properties.
function popupmenu1_CreateFcn(hObject, eventdata, handles)
% hObject    handle to popupmenu1 (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    empty - handles not created until after all CreateFcns called

% Hint: popupmenu controls usually have a white background on Windows.
%       See ISPC and COMPUTER.
if ispc && isequal(get(hObject,'BackgroundColor'),
get(0,'defaultUicontrolBackgroundColor'))
    set(hObject,'BackgroundColor','white');
end



% --- Executes on selection change in popupmenu2.
function popupmenu2_Callback(hObject, eventdata, handles)
% hObject    handle to popupmenu2 (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    structure with handles and user data (see GUIDATA)

% Hints: contents = cellstr(get(hObject,'String')) returns popupmenu2 contents
as cell array
%        contents{get(hObject,'Value')} returns selected item from popupmenu2
global silla;
silla = get(hObject,'Value')-1
assignin('base','silla',silla);
set(handles.popupmenu3,'enable','on')
if silla == 0
    axes(handles.axes4)
    imshow('velocidad_plataforma_1.jpeg')
elseif silla == 1
    axes(handles.axes4)
    imshow('velocidad_plataforma_2.jpeg')
elseif silla == 2
    axes(handles.axes4)
    imshow('velocidad_plataforma_3.jpg')
end

% --- Executes during object creation, after setting all properties.
function popupmenu2_CreateFcn(hObject, eventdata, handles)
% hObject    handle to popupmenu2 (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    empty - handles not created until after all CreateFcns called

% Hint: popupmenu controls usually have a white background on Windows.
%       See ISPC and COMPUTER.
if ispc && isequal(get(hObject,'BackgroundColor'),
get(0,'defaultUicontrolBackgroundColor'))
    set(hObject,'BackgroundColor','white');
end
```

```
% --- Executes on selection change in popupmenu3.
function popupmenu3_Callback(hObject, eventdata, handles)
% hObject    handle to popupmenu3 (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    structure with handles and user data (see GUIDATA)

% Hints: contents = cellstr(get(hObject,'String')) returns popupmenu3 contents
as cell array
%        contents{get(hObject,'Value')} returns selected item from popupmenu3
global cabeza;
cabeza = get(hObject,'Value')-1
assignin('base','cabeza',cabeza);
if get(hObject,'Value')-1 == 0
    set(handles.popupmenu5,'enable','on')
    global eje_giro;
    eje_giro = 0;
    assignin('base','eje_giro',eje_giro);
else
    set(handles.popupmenu4,'enable','on')
end

% --- Executes during object creation, after setting all properties.
function popupmenu3_CreateFcn(hObject, eventdata, handles)
% hObject    handle to popupmenu3 (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    empty - handles not created until after all CreateFcns called

% Hint: popupmenu controls usually have a white background on Windows.
%       See ISPC and COMPUTER.
if ispc && isequal(get(hObject,'BackgroundColor'),
get(0,'defaultUicontrolBackgroundColor'))
    set(hObject,'BackgroundColor','white');
end



% --- Executes on button press in pushbutton2.
function pushbutton2_Callback(hObject, eventdata, handles)
% hObject    handle to pushbutton2 (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    structure with handles and user data (see GUIDATA)
constantes_robot


% --- Executes during object creation, after setting all properties.
function axes2_CreateFcn(hObject, eventdata, handles)
% hObject    handle to axes2 (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    empty - handles not created until after all CreateFcns called

% Hint: place code in OpeningFcn to populate axes2
% axes(hObject)
% imshow('robot_foto.jpg')

% --- Executes on selection change in popupmenu4.
function popupmenu4_Callback(hObject, eventdata, handles)
% hObject    handle to popupmenu4 (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
```

```
% handles    structure with handles and user data (see GUIDATA)

% Hints: contents = cellstr(get(hObject,'String')) returns popupmenu4 contents
as cell array
%        contents{get(hObject,'Value')} returns selected item from popupmenu4
global eje_giro;
eje_giro = get(hObject,'Value')-1
assignin('base','eje_giro',eje_giro);
set(handles.popupmenu5,'enable','on')
if eje_giro == 0
    axes(handles.axes5)
    imshow('velocidad_cabeza_1.jpeg')
elseif eje_giro == 1
    axes(handles.axes5)
    imshow('velocidad_cabeza_2.jpeg')
elseif eje_giro == 2
    axes(handles.axes5)
    imshow('velocidad_cabeza_1.jpeg')
elseif eje_giro == 3
    axes(handles.axes5)
    imshow('velocidad_cabeza_3.jpg')
end

% --- Executes during object creation, after setting all properties.
function popupmenu4_CreateFcn(hObject, eventdata, handles)
% hObject    handle to popupmenu4 (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    empty - handles not created until after all CreateFcns called

% Hint: popupmenu controls usually have a white background on Windows.
%       See ISPC and COMPUTER.
if ispc && isequal(get(hObject,'BackgroundColor'),
get(0,'defaultUicontrolBackgroundColor'))
    set(hObject,'BackgroundColor','white');
end



% --- Executes on button press in checkbox1.
function checkbox1_Callback(hObject, eventdata, handles)
% hObject    handle to checkbox1 (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    structure with handles and user data (see GUIDATA)

% Hint: get(hObject,'Value') returns toggle state of checkbox1
global plot1;
plot1 = get(hObject,'Value')
assignin('base','plot1',plot1);



% --- Executes during object creation, after setting all properties.
function axes3_CreateFcn(hObject, eventdata, handles)
% hObject    handle to axes3 (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    empty - handles not created until after all CreateFcns called

% Hint: place code in OpeningFcn to populate axes3
% axes(hObject)
% imshow('robot_foto.jpg')
```

```
% --- Executes on button press in checkbox2.
function checkbox2_Callback(hObject, eventdata, handles)
% hObject    handle to checkbox2 (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    structure with handles and user data (see GUIDATA)

% Hint: get(hObject,'Value') returns toggle state of checkbox2
global plot2;
plot2 = get(hObject,'Value')
assignin('base','plot2',plot2);


function edit1_Callback(hObject, eventdata, handles)
% hObject    handle to edit1 (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    structure with handles and user data (see GUIDATA)

% Hints: get(hObject,'String') returns contents of edit1 as text
%        str2double(get(hObject,'String')) returns contents of edit1 as a double
global kp;
kp = str2double(get(hObject,'String'))
assignin('base','kp',kp);
set(handles.edit2,'enable','on')

% --- Executes during object creation, after setting all properties.
function edit1_CreateFcn(hObject, eventdata, handles)
% hObject    handle to edit1 (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    empty - handles not created until after all CreateFcns called

% Hint: edit controls usually have a white background on Windows.
%       See ISPC and COMPUTER.
if ispc && isequal(get(hObject,'BackgroundColor'),
get(0,'defaultUicontrolBackgroundColor'))
    set(hObject,'BackgroundColor','white');
end



function edit2_Callback(hObject, eventdata, handles)
% hObject    handle to edit2 (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    structure with handles and user data (see GUIDATA)

% Hints: get(hObject,'String') returns contents of edit2 as text
%        str2double(get(hObject,'String')) returns contents of edit2 as a double
global ki;
ki = str2double(get(hObject,'String'))
assignin('base','ki',ki);
set(handles.edit3,'enable','on')

% --- Executes during object creation, after setting all properties.
function edit2_CreateFcn(hObject, eventdata, handles)
% hObject    handle to edit2 (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    empty - handles not created until after all CreateFcns called
```

```
% Hint: edit controls usually have a white background on Windows.
%       See ISPC and COMPUTER.
if ispc && isequal(get(hObject,'BackgroundColor'),
get(0,'defaultUicontrolBackgroundColor'))
    set(hObject,'BackgroundColor','white');
end



function edit3_Callback(hObject, eventdata, handles)
% hObject    handle to edit3 (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    structure with handles and user data (see GUIDATA)

% Hints: get(hObject,'String') returns contents of edit3 as text
%        str2double(get(hObject,'String')) returns contents of edit3 as a double
global ki1;
ki1 = str2double(get(hObject,'String'))
assignin('base','ki1',ki1);
set(handles.pushbutton2,'enable','on')

% --- Executes during object creation, after setting all properties.
function edit3_CreateFcn(hObject, eventdata, handles)
% hObject    handle to edit3 (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    empty - handles not created until after all CreateFcns called

% Hint: edit controls usually have a white background on Windows.
%       See ISPC and COMPUTER.
if ispc && isequal(get(hObject,'BackgroundColor'),
get(0,'defaultUicontrolBackgroundColor'))
    set(hObject,'BackgroundColor','white');
end



% --- Executes during object creation, after setting all properties.
function axes4_CreateFcn(hObject, eventdata, handles)
% hObject    handle to axes4 (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    empty - handles not created until after all CreateFcns called

% Hint: place code in OpeningFcn to populate axes4



% --- Executes during object creation, after setting all properties.
function axes5_CreateFcn(hObject, eventdata, handles)
% hObject    handle to axes5 (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    empty - handles not created until after all CreateFcns called

% Hint: place code in OpeningFcn to populate axes5



% --- Executes during object creation, after setting all properties.
function axes1_CreateFcn(hObject, eventdata, handles)
% hObject    handle to axes1 (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    empty - handles not created until after all CreateFcns called
```

```
% Hint: place code in OpeningFcn to populate axes1


% --- Executes on selection change in popupmenu5.
function popupmenu5_Callback(hObject, eventdata, handles)
% hObject    handle to popupmenu5 (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    structure with handles and user data (see GUIDATA)

% Hints: contents = cellstr(get(hObject,'String')) returns popupmenu5 contents
as cell array
%        contents{get(hObject,'Value')} returns selected item from popupmenu5
global a_lineal;
a_lineal = get(hObject,'Value')-1
assignin('base','a_lineal',a_lineal);
set(handles.pushbutton1,'enable','on')

% --- Executes during object creation, after setting all properties.
function popupmenu5_CreateFcn(hObject, eventdata, handles)
% hObject    handle to popupmenu5 (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    empty - handles not created until after all CreateFcns called

% Hint: popupmenu controls usually have a white background on Windows.
%       See ISPC and COMPUTER.
if ispc && isequal(get(hObject,'BackgroundColor'),
get(0,'defaultUicontrolBackgroundColor'))
    set(hObject,'BackgroundColor','white');
end
```

8.8 modelo_vestibular.m

```
% Constantes y Variables sistema vestibular

m = 1070;   %g/cm4 Endolymph mass within the horizontal canal

d = 0.76;   %g/cm Inertial forcing coefficient

tau1 = 13.2;    % Slow time constant of the horizontal semicircular canal

tau2 = 0.006;    % Fast time constant of the horizontal semicircular canal

%taua = 80;

%taui = 0.049;

% Selección del tipo de función velocidad angular de la silla (constante,
% trapezoidal, senoidal)
silla = 2;

% ¿ Considerar movimiento de la cabeza ?
cabeza = 1;

% Selección tipo de función velocidad de la cabeza
eje_giro = 3;

% Selección del tipo de canales semicirculares (ortogonal-> canal=0,
```

```
% no-ortogonal-> canal=1)
canal = 1;

% Selección aceleración lineal de la cabeza (sin aceleracion 0 con
% aceleración 1)
a_lineal = 0;

% Simulaciones y plots de resultados
sim('vestibular_system_final')

% Canales semicirculares
figure
subplot(3,1,1)
plot(t,desp_cupula_anterior,'lineWidth',2)
title('Canal anterior (cm^3)')
xlabel('t(s)')
subplot(3,1,2)
plot(t,desp_cupula_posterior,'lineWidth',2)
title('Canal posterior (cm^3)')
xlabel('t(s)')
subplot(3,1,3)
plot(t,desp_cupula_lateral,'lineWidth',2)
title('Canal lateral (cm^3)')
xlabel('t(s)')

% Otolitos
figure(2)
subplot(3,1,1)
plot(t,x_otolito,'lineWidth',2)
title('Afferent Firing Rate (AFR) (eje X)')
xlabel('t(s)')
subplot(3,1,2)
plot(t,y_otolito,'lineWidth',2)
title('Afferent Firing Rate (AFR) (eje Y)')
xlabel('t(s)')
subplot(3,1,3)
plot(t,z_otolito,'lineWidth',2)
title('Afferent Firing Rate (AFR) (eje Z)')
xlabel('t(s)')
```

8.9 robot_simulacion.m

```
% Robot con control de posición mediante canal semicircular

% Creación del modelo del motor de corriente continua
DCmotor

%Parámetros del controlador
ki = 8;
kp = 2;
ki1 = 5;

% Simulación del robot
open_system('vestibular_3GDL')
sim('vestibular_3GDL')

% Plots
figure
subplot(3,1,1)
plot(t,gdl1,'lineWidth',2)
```

```
title('posición angular eje Z (grados)')
xlabel('t(s)')
subplot(3,1,2)
plot(t,gdl2,'lineWidth',2)
title('posición angular eje X (grados)')
xlabel('t(s)')
subplot(3,1,3)
plot(t,gdl3,'lineWidth',2)
title('posición angular eje Y (grados)')
xlabel('t(s)')
```

8.10 DCmotor.m

```
% DC motor model

R = 2.0;                % Ohms (2 ohmios la original)
L = 0.5;                % Henrys
Km = 0.1;               % torque constant
Kb = 0.1;               % back emf constant
Kf = 0.2;               % Nms
J = 0.02;               % kg.m^2/s^2
% Td = -0.1 * (t>5 & t<10);       % load disturbance

% Construcción del modelo

h1 = tf(Km,[L R]);            % armature
h2 = tf(1,[J Kf]);            % eqn of motion

dcm = ss(h2) * [h1 , 1];      % w = h2 * (h1*Va + Td)
dcm = feedback(dcm,Kb,1,1);   % close back emf loop

Kff = 1/dcgain(dcm(1));        % ganancia del sistema

% stepplot(dcm(1));

sim ('DCmotormodel')
```